# CareerSphere AI: An Explainable, Skill-Aware Campus Placement Platform with Opportunity Math, Skill Roadmaps and a Grounded AI Copilot

*A Report submitted in partial fulfillment of the requirements*
*for the Degree of*

**Bachelor of Technology**

in

**COMPUTER SCIENCE AND ENGINEERING**

by


| | |
|---|---|
| **Venigella Venkata Rupin** | **(2311CS010689)** |
| **Elala Manideep Reddy** | **(2311CS010182)** |
| **Jalla Pruthvi Sai** | **(2311CS010264)** |
| **Syed Jaffar Ali** | **(2311CS010639)** |


*Under the guidance of*

**Mr. D. Chinna Kasi Reddy**

[Designation], Department of CSE




**Department of Computer Science and Engineering**

**School of Engineering**

**MALLA REDDY UNIVERSITY**

**Maisammaguda, Dulapally, Hyderabad, Telangana 500100**


**2026**

# CareerSphere AI: An Explainable, Skill-Aware Campus Placement Platform with Opportunity Math, Skill Roadmaps and a Grounded AI Copilot

*A Major Report submitted in partial fulfillment of the requirements for the Degree of*

**Bachelor of Technology**

in

**COMPUTER SCIENCE AND ENGINEERING**

by


| | |
|---|---|
| **Venigella Venkata Rupin** | **(2311CS010689)** |
| **Elala Manideep Reddy** | **(2311CS010182)** |
| **Jalla Pruthvi Sai** | **(2311CS010264)** |
| **Syed Jaffar Ali** | **(2311CS010639)** |


*Under the guidance of*

**Mr. D. Chinna Kasi Reddy**

[Designation], Department of CSE


**Department of Computer Science and Engineering**

**School of Engineering**

**MALLA REDDY UNIVERSITY**

**Maisammaguda, Dulapally, Hyderabad, Telangana 500100**


**2026**

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

**Department of Computer Science and Engineering**

# CERTIFICATE

This is to certify that the project report entitled **"CAREERSPHERE AI: AN EXPLAINABLE, SKILL-AWARE CAMPUS PLACEMENT PLATFORM WITH OPPORTUNITY MATH, SKILL ROADMAPS AND A GROUNDED AI COPILOT"** submitted by Venigella Venkata Rupin (2311CS010689), Elala Manideep Reddy (2311CS010182), Jalla Pruthvi Sai (2311CS010264) and Syed Jaffar Ali (2311CS010639) towards the partial fulfillment for the award of Bachelor's Degree in Computer Science and Engineering, Malla Reddy University, Hyderabad, is a record of bonafide work done by them. The results embodied in the work are not submitted to any other University or Institute for award of any degree or diploma.

**INTERNAL GUIDE**

**Mr. D. Chinna Kasi Reddy**

**Designation, CSE**

**DEAN-CSE**

**Dr. A.V.L.N. Sujith**

**External Examiner**

MALLA REDDY UNIVERSITY
(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

**Department of Computer Science and Engineering**

# DECLARATION

We hereby declare that the project report entitled **"CAREERSPHERE AI: AN EXPLAINABLE, SKILL-AWARE CAMPUS PLACEMENT PLATFORM WITH OPPORTUNITY MATH, SKILL ROADMAPS AND A GROUNDED AI COPILOT"** has been carried out by us and this work has been submitted to the Department of Computer Science and Engineering, Malla Reddy University, Hyderabad in partial fulfillment of the requirements for the award of degree of Bachelor of Technology. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions. All data used in this report is fictional demo data created for the project.

**Place** Hyderabad

**Date** DD-MM-2026

| | |
|---|---|
| **Venigella Venkata Rupin** | **(2311CS010689)** |
| **Elala Manideep Reddy** | **(2311CS010182)** |
| **Jalla Pruthvi Sai** | **(2311CS010264)** |
| **Syed Jaffar Ali** | **(2311CS010639)** |

# ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the successful completion of this project report. Firstly, we would like to express our heartfelt thanks to **Mr. Chamakura Malla Reddy, Chairman**, and **Dr V. S. K. Reddy, Vice-Chancellor**, for their visionary leadership and unwavering commitment to academic excellence.

We would also like to express our deepest appreciation to our project guide, **Mr. D. Chinna Kasi Reddy, Designation, Department of Computer Science and Engineering**, whose invaluable guidance, insightful feedback and constant support have been instrumental throughout the course of this project.

We sincerely thank **Dr. A.V.L.N. Sujith, Dean, CSE, School of Engineering**, for his encouragement and support throughout our academic journey, and the Training and Placement Cell of the university for explaining how campus placement works in practice, including the placement portal that shaped the placement profile in this project.

We are deeply indebted to all of them for their support, encouragement and guidance, without which this project would not have been possible.

| | |
|---|---|
| **Venigella Venkata Rupin** | **(2311CS010689)** |
| **Elala Manideep Reddy** | **(2311CS010182)** |
| **Jalla Pruthvi Sai** | **(2311CS010264)** |
| **Syed Jaffar Ali** | **(2311CS010639)** |

# ABSTRACT

Campus placement portals record profiles, applications and results, but rarely tell anyone what to do next: which skill a student should learn, which students a recruiter should look at, or which training the placement cell should run. This project designs, implements and evaluates **CareerSphere AI**, a placement platform for students, recruiters and the placement cell built on one transparent, level-aware match rule: the weighted share of a job's required skills a student has, where must-have skills count twice as much as nice-to-have ones and a skill below the level asked earns half credit, with a 70% "within reach" threshold and the recruiter's CGPA cutoff. Skill levels can be verified by server-graded assessments or by certificates checked by the placement cell.

Four decision tools are built on the rule. **Opportunity Math** ranks each learning step by how many open jobs it would bring within reach. A **Skill Roadmap** searches every bundle of up to three steps exactly and marks steps that only pay off together. **Hidden Talent** shows recruiters consented students who fit a job but have not applied. A **Workshop Planner** chooses k workshops for the batch by exact bit-mask search, because the objective is not submodular and greedy selection can fail. A Copilot (Google Gemini with eight read-only tools) and an evidence-checked résumé skill finder are grounded in the same data and work without an AI key. The system uses Spring Boot, React and MySQL, runs from Terminal, Xcode, VS Code and Android Studio and online with Docker, and has 39 unit tests and 17 API tests.

On a fictional batch of 35 students and 12 jobs, Opportunity Math brings 1.43 jobs per student within reach against 1.03 for the most-demanded missing skill; the Skill Roadmap raises this to 2.89 with two steps and 4.71 with three, helping all 7 students for whom no single skill helps; Hidden Talent surfaces 10 fitting non-applicants; and three exactly planned workshops give all 35 students a job within reach where greedy selection reaches 32, in 6.5 ms.



**Keywords:** campus placement, skill-gap analysis, explainable recommendation, combinatorial optimisation, large language models, Spring Boot, React.

# TABLE OF CONTENTS


| DESCRIPTION | PAGE NUMBER |
|---|---|
| ABSTRACT | VI |
| Chapter 1 Introduction | 1 |
| 1.1 Project Overview | 1 |
| 1.2 Problem Statement | 2 |
| 1.3 Scope and Research Questions | 3 |
| 1.4 Feasibility Analysis | 4 |
| Chapter 2 Literature Survey | 5 |
| 2.1 Foundations of Matching, Explanation and Coverage | 5 |
| 2.2 Related Work and Existing Systems | 6 |
| 2.3 Research Gap and Rationale for the Proposed System | 7 |
| Chapter 3 Project Objectives And Requirements | 8 |
| 3.1 Project Objectives | 8 |
| 3.2 Limitations and Scope Boundaries | 9 |
| 3.3 Functional and Non-Functional Requirements | 10 |
| 3.4 Hardware and Software Requirements | 11 |
| Chapter 4 Design And Methodology | 12 |
| 4.1 Proposed System Framework | 12 |
| 4.2 System Architecture | 13 |
| 4.3 Functional Module Description | 15 |
| 4.4 Methods and Mathematical Formulation | 16 |
| 4.5 Database Design | 19 |
| 4.6 API Design | 20 |
| 4.7 Data Flow Diagrams (Levels 0, 1 and 2) | 21 |
| 4.8 UML Design Models | 22 |
| Chapter 5 Testing | 30 |
| 5.1 Need for Testing and Verification Strategy | 30 |
| 5.2 Unit Testing of the Rules | 31 |
| 5.3 Integration and API Testing | 32 |
| 5.4 User Interface Testing and Validation | 33 |
| Chapter 6 Code And Implementation | 34 |
| 6.1 Technology Stack and Project Structure | 34 |
| 6.2 Core Source Code | 36 |
| 6.3 System Execution Logs and Output Verification | 38 |
| 6.4 Screens of the Application | 39 |
| 6.5 Running on Different Platforms | 50 |

# TABLE OF CONTENTS CONTINUED


| DESCRIPTION | PAGE NUMBER |
|---|---|
| **Chapter 7 Results And Discussion** | **51** |
| **7.1 Experimental Results** | **51** |
| **7.2 Discussion and Evaluation of the Research Questions** | **55** |
| **7.3 Comparison with Existing Approaches** | **56** |
| **Chapter 8 Conclusion And Future Scope** | **57** |
| **8.1 Conclusions and Research Contributions** | **57** |
| **8.2 Future Scope** | **58** |
| **REFERENCES** | **59** |
| **APPENDIX SETUP COMMANDS, DEMO SCRIPT AND API LIST** | **61** |

# LIST OF FIGURES

**FIG NO FIG NAME PAGE NO**


Figure 4.1 Framework of CareerSphere AI from inputs through the match rule to decisions and evaluation 12

Figure 4.2 Layered architecture of CareerSphere AI 14

Figure 4.3 Entity–relationship diagram (rotated to fit the page) 19

Figure 4.4 Data flow diagrams at Level 0 (context), Level 1 (processes) and Level 2 (matching and planning) 21

Figure 4.5 Use case diagram of CareerSphere AI 22

Figure 4.6 Class diagram of the principal entities and services 23

Figure 4.7 Sequence diagram of login and an authenticated request 24

Figure 4.8 Sequence diagram of applying to a job 25

Figure 4.9 Sequence diagram of the Copilot with function calling 25

Figure 4.10 Sequence diagram of the evidence-checked résumé skill finder 26

Figure 4.11 Sequence diagram of building a Skill Roadmap 26

Figure 4.12 Activity diagram of the student placement flow 27

Figure 4.13 Deployment diagram of the local and online environments 28

Figure 4.14 Flow of the exact Workshop Planner 29

Figure 6.1 Mind map of the project: features, algorithms and technologies 35

Figure 6.2 Login page with demo accounts for the three roles 39

Figure 6.3 Career Map: each open job placed by match score; skill levers on the right 40

Figure 6.4 Career Map with the Copilot panel open 40

Figure 6.5 Opportunity Math: the skills that bring the most jobs within reach 41

Figure 6.6 Skill Roadmap: best one-, two- and three-step plans 41

Figure 6.7 Job listings with explained match scores 42

Figure 6.8 Job detail: why this score, and the job’s requirements 42

Figure 6.9 Placement profile of a new student: guided steps and résumé score 43

Figure 6.10 Placement profile of a student who has completed most steps 43

Figure 6.11 Skill assessments: verified and unverified skills 44

Figure 6.12 A timed assessment in progress 44

Figure 6.13 Assessment result: fewer than 4 correct verifies nothing; retake after 24 hours 45

Figure 6.14 Certifications: a verified certificate and how verification works 45

Figure 6.15 Readiness leaderboard 46

Figure 6.16 Recruiter: posting a job with skill levels and must-have or nice-to-have flags 46

Figure 6.17 Recruiter: applicants ranked by match, with verified skills 47

Figure 6.18 Recruiter: Hidden Talent for a job, with invitations 47

Figure 6.19 Placement cell: analytics and the curriculum–market skill gap 48

Figure 6.20 Placement cell: Workshop Planner for three workshops 48

Figure 6.21 Placement cell: certificates waiting for verification 49

Figure 6.22 The same app on a phone: Career Map and the slide-in menu 49
Figure 7.1 Mean new jobs within reach per student for each way of choosing what to learn 52
Figure 7.2 Outcome of each planning method for k = 1 to 4 workshops; dotted line: today. (a) Reach goal; (b) Matches goal 53
Figure 7.3 Exact Workshop Planner runtime against the number of workshops (logarithmic scale) 54
Figure 7.4 Hidden Talent per job: consented students who have not applied, within reach or one step away 54

# LIST OF TABLES


| TABLE NO | TABLE NAME | PAGE NO |
|---|---|---|
| Table 2.1 | Typical placement portal compared with CareerSphere AI | 6 |
| Table 3.1 | Functional requirements by role | 10 |
| Table 3.2 | Hardware and software configuration | 11 |
| Table 4.1 | Modules of the system | 15 |
| Table 4.2 | Résumé score rules | 18 |
| Table 4.3 | Main REST endpoints | 20 |
| Table 5.1 | Unit tests | 31 |
| Table 5.2 | End-to-end API test cases | 32 |
| Table 6.1 | Technology stack | 34 |
| Table 7.1 | Recommended next skill, 35 students | 51 |
| Table 7.2 | Skill Roadmap, 35 students | 51 |
| Table 7.3 | Workshop plans: students reached / matches (Reach goal) | 52 |
| Table 7.4 | Planner runtime, median of 9 calls | 53 |
| Table 7.5 | Comparison of existing approaches with CareerSphere AI | 56 |
| Table A.1 | Demo accounts | 61 |
| Table A.2 | All REST endpoints | 62 |

# ABBREVIATIONS AND NOTATION

**API** Application Programming Interface

**BCrypt** Blowfish-based adaptive password hashing function

**CGPA** Cumulative Grade Point Average

**CSS** Cascading Style Sheets

**DFD** Data Flow Diagram

**DTO** Data Transfer Object

**ER** Entity–Relationship

**F1** Harmonic mean of precision and recall

**H2** In-memory Java SQL database used for tests

**HTTPS** Hypertext Transfer Protocol Secure

**JPA** Jakarta Persistence API

**JSON** JavaScript Object Notation

**JWT** JSON Web Token

**LLM** Large Language Model

**REST** Representational State Transfer

**SPA** Single-Page Application

**SQL** Structured Query Language

**UI** User Interface

**UML** Unified Modeling Language

**UUID** Universally Unique Identifier

$\ell_s(x)$ Level of student s in skill x (0 none, 1 Beginner, 2 Intermediate, 3 Advanced)

$m_j(x)$, $w_j(x)$ Minimum level and weight (2 must-have, 1 nice-to-have) of skill x in job j

$\kappa$ Credit of one skill: 1, ½ or 0

$k$ Number of workshops in a plan

# CHAPTER-1

# INTRODUCTION

## 1.1 Project Overview

Campus placement is the main route from an engineering college to a first job in India. Within a few months each year a training and placement cell invites companies, students apply to openings, recruiters shortlist and interview, and offers are made. The process is high in volume and short in time: a mid-sized department may have several hundred final-year students and dozens of openings, each asking for its own mix of technical skills, a minimum CGPA and sometimes a particular branch.

Most colleges run this process on a placement portal, a website where students keep a profile and résumé, companies post drives and the placement cell tracks applications. These portals are good at recording what happened and weak at telling anybody what to do next. A student can see that a job asks for Java, Spring Boot and Git, but not whether learning Spring Boot or Docker would open more doors across all current openings. A recruiter sees a list of applicants, but not the students who would fit well and simply did not apply. A placement cell can see that many students lack a skill, but not which three workshops would help the most students reach at least one job.

This project, **CareerSphere AI**, is a campus placement platform for students, recruiters and the placement cell built around one transparent rule: how much of a job's required skill set a student has, with must-have skills counting twice as much as nice-to-have ones and a skill below the level asked earning half credit. Because the rule is simple and exact, it can be evaluated thousands of times a second to answer "what if" questions for every user, and every answer can be explained line by line. On top of the rule the system offers Opportunity Math (the best next skill), a Skill Roadmap (the best two- and three-step learning plans), Hidden Talent (fitting students who did not apply), an exact Workshop Planner for the batch, verified skill levels, and an AI Copilot that is grounded in the same data.

## 1.2 Problem Statement

Existing placement systems either show a score and a list of missing skills, or predict whether a student will be placed. Neither tells a student which action to take, and neither helps the placement cell plan training for the whole batch. Advice that looks at one skill at a time can also be wrong in a way that matters, because skills pay off in combination: a student who has only Git gains nothing from learning Java alone or Spring Boot alone for a job that needs all three, but gains the job by learning both.

A second problem is trust. Skill claims on most portals are self-reported, and AI features, where they exist, can produce statements that nobody can check. A placement decision should not depend on a statement that cannot be verified.

The problem addressed in this project is therefore to design and build a placement platform that (i) scores every student against every open job with a transparent, level-aware rule; (ii) turns that score into concrete, explainable decisions for students, recruiters and the placement cell, treating skill combinations exactly; and (iii) uses large language models only where their output can be grounded in the same data.

## 1.3 Scope and Research Questions

The investigation is organised around four research questions that decide what is measured and how the results are read.

- **RQ1 – Next skill.** How many more open jobs does the action chosen by Opportunity Math bring within reach of a student, compared with recommending the missing skill that most jobs ask for?
- **RQ2 – Multi-step plans.** How much does an exact search over plans of two and three learning steps add to the best single step, and how often do those plans depend on skills that only pay off together?
- **RQ3 – Batch planning.** Does exact search over workshop combinations reach more students than greedy, one-at-a-time selection and simple demand-based rankings, and is it fast enough to use interactively?
- **RQ4 – Recruiter reach.** How many consented students fit a job, or are one step away from it, without having applied?

The system covers the core placement cycle inside one college: profiles, résumés, skills and levels, jobs, applications, shortlisting and offers, plus the guidance and planning features. All numbers in this report come from a fictional demo batch shipped with the system (35 students, 2 companies, 12 jobs, 22 skills), so they show how the methods behave rather than their effect on real placements.

## 1.4 Feasibility Analysis

Before development, the project was assessed from four viewpoints.

1. **Technical feasibility.** Every component uses mature, free and widely taught technology: Java 17 with Spring Boot, React and MySQL. The exact searches are small (about 1,000 learning plans per student and 1,540 workshop plans for k = 3) and run in milliseconds on a laptop.
2. **Operational feasibility.** The workflow follows the steps students and placement officers already know from the university placement portal (résumé, consent, assessment, video), and every screen explains its numbers in plain language. Demo accounts and seeded data let a new user explore all three roles at once.
3. **Economic feasibility.** The software stack has no licence cost. Development and demonstration run on a student laptop; online hosting on a platform such as Railway costs a few dollars a month, and the system can equally run on a college server with Docker.
4. **AI feasibility.** The AI features use the free tier of the Google Gemini API and fall back to rule-based answers when no key is set, so the system never depends on an external service to work.

# CHAPTER-2

# LITERATURE SURVEY

## 2.1 Foundations of Matching, Explanation and Coverage

**Job recommendation and person–job fit.** De Ruijt and Bhulai [1] review a decade of job recommender systems and group methods into content-based, collaborative, hybrid and knowledge-based approaches. They note that job recommendation differs from product recommendation: it is reciprocal (both the candidate and the employer must agree), temporal (openings close), and has received limited attention to fairness. Zhu et al. [2] learn joint neural representations of résumés and job postings to predict person–job fit. Such models can be accurate when large labelled histories exist, but a college placement cell rarely has that data, and a learned score offers little explanation of why a particular student fits a particular job.

**Explainable recommendation.** Zhang and Chen [4] distinguish post-hoc explanations, generated after a black-box model makes a prediction, from models that are interpretable by construction. CareerSphere is of the second kind: every score is computed from the listed skills and levels, so the explanation (matched skills, missing skills, level gaps, CGPA rule) is the computation itself.

**Coverage and submodularity.** Choosing k items to cover as many users as possible is a maximum coverage problem. For monotone submodular objectives, where adding an item helps less the more items have already been chosen, the greedy algorithm achieves a $1 - 1/e$ (about 63%) approximation [5], which is the best possible in polynomial time unless P = NP [6]; Krause and Golovin [7] survey the area. Workshop and learning-plan selection in this project has a threshold (a job is within reach at 70% of its skills), which creates complementarities between skills and breaks submodularity, so the greedy guarantee does not apply.

## 2.2 Related Work and Existing Systems

**Placement prediction.** Alfana et al. [3] build probability models from cognitive and academic features to predict campus placement. Prediction is useful for planning, but it describes a situation rather than changing it: a 40% chance of placement does not say which skill to learn.

**Tool use and grounding in language models.** Toolformer [8] and ReAct [9] show that language models can decide when to call external tools and use the results in their answers; commercial APIs expose the same pattern as function calling [16]. Ji et al. [10] survey hallucination, where a model states things not supported by its input, and Gao et al. [11] show that even strong models often fail to support their statements with correct citations.

**Fairness in algorithmic hiring.** Raghavan et al. [12] examine the claims of vendors of algorithmic pre-employment assessments and the practical limits of removing bias. CareerSphere uses no demographic attributes; its inputs are skills, levels and the recruiter's published CGPA cutoff, and every score can be audited by hand.

**University placement portals.** Portals such as the one used at our university offer student profiles, résumé upload, placement consent, skill assessments, certificate upload with verification, an introduction video, drive listings and a résumé score. CareerSphere includes these features and adds the decision-support layer (Table 2.1).

**Table 2.1 Typical placement portal compared with CareerSphere AI**

| Feature | Typical portal | CareerSphere AI |
|---|---|---|
| Profile, résumé, consent, video | Yes | Yes, as guided steps |
| Skill assessment | Often | Yes, server-graded; sets a verified level |
| Certificate verification | Often | Yes, by the placement cell |
| Match score for a job | Rarely, or opaque | Explained line by line, level-aware |
| Best next skill across all jobs | No | Opportunity Math |
| Best two- and three-step learning plan | No | Skill Roadmap (exact search) |
| Recruiter sees fitting non-applicants | No | Hidden Talent with invitations |
| Workshop planning for the batch | No | Exact Workshop Planner |
| AI assistant | Rarely | Copilot grounded in 8 read-only tools |
| AI résumé reading | Rarely | Evidence-checked skill finder |

## 2.3 Research Gap and Rationale for the Proposed System

The literature offers accurate but opaque fit models, outcome predictors, and general results on coverage and on tool-using language models. What is missing for a college is a system that connects a transparent score to decisions at three levels (student, recruiter, placement cell), recognises that skills pay off in combination and searches those combinations exactly, and uses language models only where their output can be checked. CareerSphere is designed to fill this gap. Its new elements are Opportunity Math, the Skill Roadmap with synergy detection, Hidden Talent, the exact Workshop Planner, verified skill levels, and grounded AI.

# CHAPTER-3

# PROJECT OBJECTIVES AND REQUIREMENTS

## 3.1 Project Objectives

The engineering and research goals of the project are the following.

1. **Core platform.** Build a working web platform for three roles (student, recruiter, placement cell) with secure login, profiles, résumé upload, job posting, applications and status tracking.
2. **Explainable match rule.** Define a level-aware match score with must-have and nice-to-have skills, and a clear "within reach" rule that includes the recruiter's CGPA cutoff.
3. **Student guidance.** Provide Opportunity Math, a what-if Career Map, and a Skill Roadmap that finds the best two- and three-step plans exactly.
4. **Recruiter tools.** Rank applicants with explanations and verified skills, and show Hidden Talent: consented students who fit but have not applied.
5. **Placement-cell tools.** Provide batch analytics, certificate verification, a readiness leaderboard and an exact Workshop Planner.
6. **Verified levels.** Verify skill levels through server-graded assessments and placement-cell-checked certificates.
7. **Grounded AI.** Add a Copilot and a résumé skill finder that use only the student's real data and keep working without an AI key.
8. **Evaluation.** Test the system with automated unit and API tests, and evaluate the methods on a realistic demo batch against simple baselines.
9. **Portability.** Make the project run from Terminal with one command, from Xcode, VS Code and Android Studio, and online with Docker.

### 3.2 Limitations and Scope Boundaries

To keep the work focused, the following limits were set.

1. **Fictional data.** All results use a demo batch; no real student records are used, in line with the AI provider's free-tier terms.
2. **Fixed weights.** The weights (2 and 1), the half credit and the 70% threshold are design choices, not learned from data.
3. **Learning model.** A workshop is assumed to teach its skill to every invitee, and effort in the roadmap is counted in levels, not hours.
4. **Out of scope.** Interview scheduling with calendars, offer letters and communication with companies outside the platform.

## 3.3 Functional and Non-Functional Requirements

**Table 3.1 Functional requirements by role**

| ID | Role | Requirement |
|---|---|---|
| FR-1 | All | Register (student or recruiter), log in and log out; admin accounts cannot be self-registered |
| FR-2 | Student | Maintain a profile: branch, CGPA, marks, links, headline, skills with levels |
| FR-3 | Student | Upload a résumé (PDF, DOC, DOCX up to 5 MB); have skills suggested from it with evidence |
| FR-4 | Student | Give or withdraw placement consent; record or upload a one-minute introduction video |
| FR-5 | Student | Take a timed skill assessment; upload certificates for verification |
| FR-6 | Student | See every open job with its match score and explanation; apply once per job |
| FR-7 | Student | See the Career Map, Opportunity Math, what-if results and the Skill Roadmap |
| FR-8 | Student | Track application status; see recruiter invitations and the readiness leaderboard |
| FR-9 | Student | Ask the Copilot questions about jobs, skills and applications |
| FR-10 | Recruiter | Post, edit, close and delete jobs with skill levels and must-have or nice-to-have flags |
| FR-11 | Recruiter | See applicants ranked by match with verified skills, certificates, résumé and video; update status |
| FR-12 | Recruiter | See Hidden Talent for a job and invite students to apply |
| FR-13 | Admin | See placement analytics, the skill demand–supply gap and the batch leaderboard |
| FR-14 | Admin | Verify or reject certificates with a note; plan k workshops and see whom to invite |

The non-functional requirements are: **security** (BCrypt password hashes, stateless JWT login, every API restricted to one role on the server, ownership checks, uploads checked by size and file signature); **privacy** (recruiters see a student only after placement consent; the leaderboard shows other students by first name and initial; the AI key stays on the server); **performance** (every page answers in well under a second on the demo data); **explainability** (every number has a sentence saying how it was computed); **reliability** (the system works fully without an AI key; duplicate applications and invitations are prevented by database constraints); and **portability** (responsive layout down to phone width; browser, Mac, iPhone and Android).

## 3.4 Hardware and Software Requirements

The development and test environment is listed in Table 3.2.

**Table 3.2 Hardware and software configuration**

| Component | Minimum | Used in development |
|---|---|---|
| Processor | Dual-core, 2 GHz | Apple M-series / Intel Core i5 |
| Memory | 4 GB RAM | 8 GB RAM or more |
| Disk | 2 GB free | 5 GB free |
| Operating system | macOS, Windows or Linux | macOS 14 or later |
| Java | JDK 17 | OpenJDK 17 (Homebrew) |
| Node.js | Node 18 | Node 20 LTS |
| Database | MySQL 8 | MySQL 8.4 (Homebrew) |
| Build tools | Maven 3.9, npm | Maven (Homebrew), npm |
| Editors | Any | VS Code, Xcode 15+, Android Studio |
| Browser | Chrome, Safari, Edge, Firefox | Chrome, Safari |
| AI (optional) | Gemini API key | Gemini free tier, demo data only |

# CHAPTER-4

# DESIGN AND METHODOLOGY

## 4.1 Proposed System Framework

The framework separates the inputs, the single match rule, and the decisions built on it (Fig 4.1). Profiles, skill levels, résumés, certificates and jobs enter the system; assessments and certificate reviews turn self-reported levels into verified ones; the match rule scores every student against every open job; and four groups of decisions are computed from those scores for students, recruiters, the placement cell and the AI assistant. Because all decisions use the same rule, a student, a recruiter and the placement cell always see consistent numbers, and the whole pipeline can be evaluated against simple baselines on the same data.

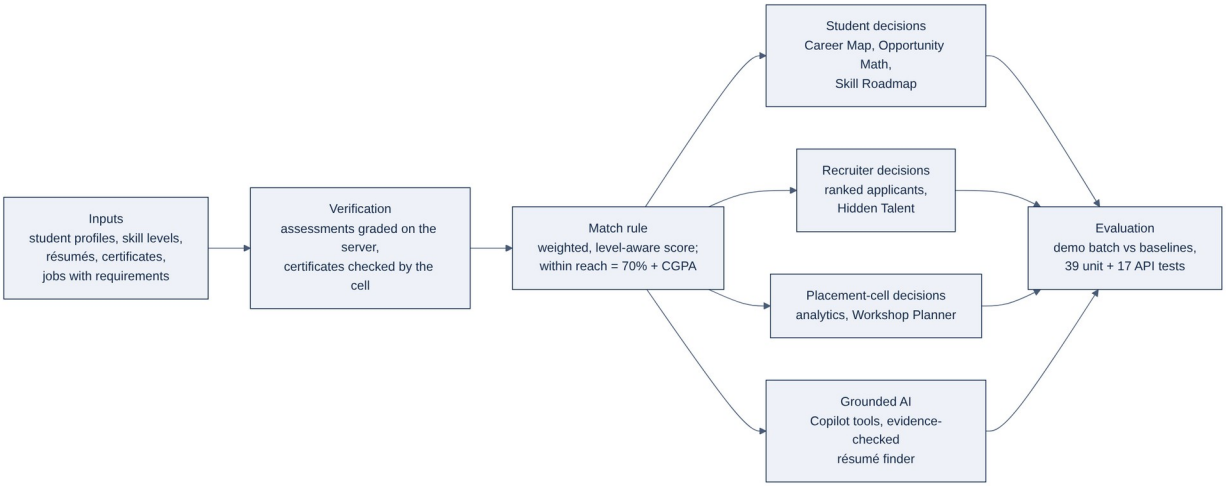



**Fig 4.1 Framework of CareerSphere AI from inputs through the match rule to decisions and evaluation**

## 4.2 System Architecture

CareerSphere follows a three-tier client–server architecture (Fig 4.2). The client is a single-page React application, used directly in a browser or wrapped by the Mac, iPhone and Android apps. The server is a Spring Boot application that exposes a REST API under `/api`. Every request passes through Spring Security, which validates the JSON Web Token and the user's role before a controller is reached. Controllers call services, which hold the business rules, and services read and write MySQL through Spring Data JPA repositories. Uploaded files are stored on disk under random names.

Two layers deserve special mention. The **rule engine** (MatchService, RoadmapService, TalentService and WorkshopService) computes every score and plan from the same rule. The **AI layer** never writes data on its own: the Copilot reads data only through tools that call the same services, and the résumé skill finder only proposes skills that the student must confirm. The Gemini API key is held by the server and never sent to the browser.

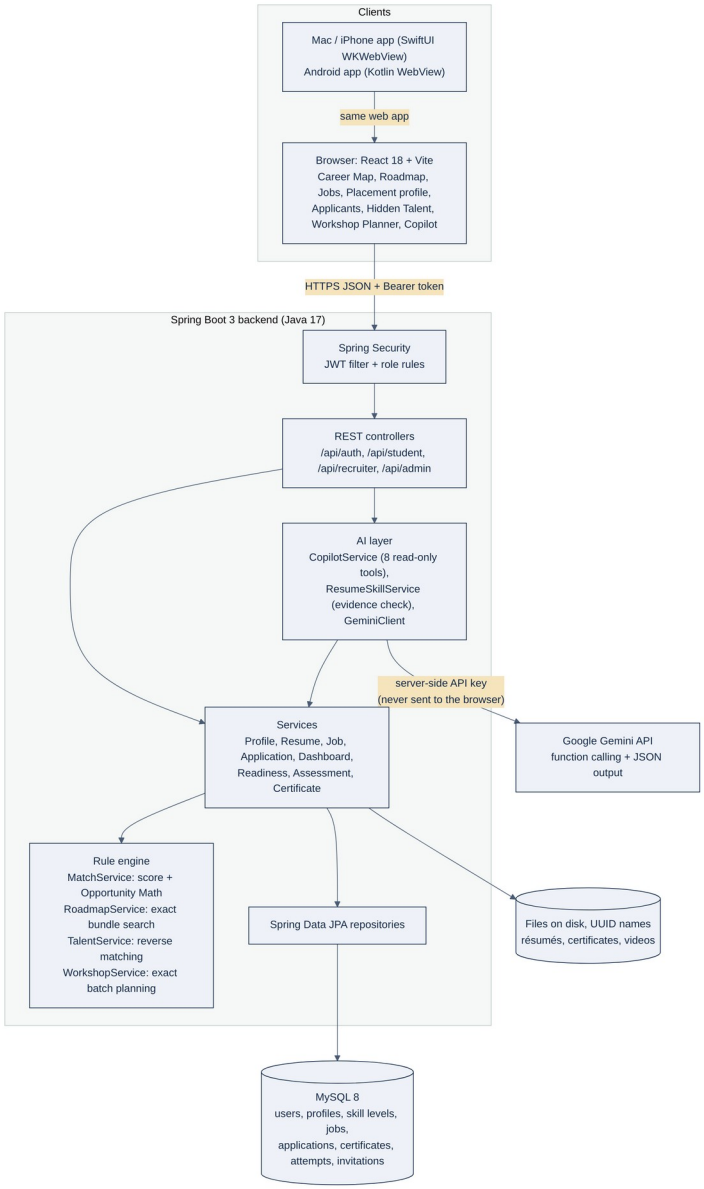



**Fig 4.2 Layered architecture of CareerSphere AI**

## 4.3 Functional Module Description

The code is divided into the loosely coupled modules listed in Table 4.1. Each module has its own endpoints, services and pages, and all modules share the match rule.

**Table 4.1 Modules of the system**

| Module | Responsibilities | Main classes and pages |
|---|---|---|
| Authentication | Register, log in, JWT issue and check, role rules | AuthService, JwtService, SecurityConfig; Login |
| Student profile | Profile, skills and levels, résumé, consent, video, résumé score | StudentProfileService, ResumeService, ReadinessService; PlacementProfile |
| Skill verification | Timed assessments, certificate upload and review | AssessmentService, QuestionBank, CertificateService; Assessments, Certifications |
| Jobs and applications | Post jobs with levels, apply, track status, rank applicants | JobService, ApplicationService; StudentJobs, JobForm, Applicants |
| Guidance | Match score, Career Map, Opportunity Math, what-if, Skill Roadmap, next best action | MatchService, DashboardService, RoadmapService; CareerMap, Opportunities, Roadmap |
| Hidden Talent | Consented non-applicants who fit; invitations | TalentService, Invitation; Applicants page |
| Placement cell | Analytics, workshop planning, certificate review, leaderboard | WorkshopService, AdminController; AdminDashboard, WorkshopPlanner |
| AI | Copilot with tools; résumé skill finder | CopilotService, CopilotTools, ResumeSkillService, GeminiClient |

## 4.4 Methods and Mathematical Formulation

All methods use integer arithmetic so that the planner, the roadmap and the explanation shown to the user never disagree by rounding.

**(a) Explainable match score.** Let a student $s$ have level $\ell_s(x) \in \{0, 1, 2, 3\}$ for each skill $x$ (0 means not known; 1 to 3 are Beginner, Intermediate and Advanced) and CGPA $g_s$. Let an open job $j$ require the skill set $R_j$, with a minimum level $m_j(x)$ for each skill, a weight $w_j(x) = 2$ for a must-have and 1 for a nice-to-have skill, and an optional CGPA cutoff $c_j$. The credit for one skill and the score are

$$\kappa(\ell, m) = 1 \text{ if } \ell \geq m; \quad \tfrac{1}{2} \text{ if } 0 < \ell < m; \quad 0 \text{ if } \ell = 0 \tag{4.1}$$

$$\text{score}(s, j) = \text{round}\left( 100 \cdot \Sigma_x \, w_j(x) \cdot \kappa(\ell_s(x), m_j(x)) \,/\, \Sigma_x \, w_j(x) \right) \tag{4.2}$$

with both sums over the required skills $x \in R_j$. In code the sums are kept in half units, and the percentage is computed as (200 · half + total) div (2 · total), which rounds halves up without floating point. When every skill is a must-have at Beginner level, (4.2) reduces to the plain share of required skills. *Example:* a job needs Java at Intermediate (must-have), Spring Boot (must-have) and Git (nice-to-have); a student has Java at Beginner, Spring Boot and Git. Credit in half units is 2 + 4 + 2 = 8 of 10, so the score is 80%, and the explanation names Java as a level gap.

**(b) Within reach.** A job is within reach when

$$\text{reach}(s, j) = [\ \text{score}(s, j) \geq 70\ ] \wedge [\ g_s \geq c_j\ ] \tag{4.3}$$

With default jobs the threshold admits 3 of 4 and 4 of 5 required skills but not 2 of 3 (67%). The CGPA rule is reported separately and never changes the score.

**(c) Opportunity Math.** An *action* is learning a skill $x$ to a level $t$, or raising a known skill to a level $t$ that some open job asks for. With $L_s$ the student's levels and reach($L$, $j$) equal to (4.3) evaluated with levels $L$, the gain of an action is

$$\text{gain}_s(x, t) = |\ \{\ j : \neg\text{reach}(L_s, j) \wedge \text{reach}(L_s \oplus (x, t), j)\ \}\ | \tag{4.4}$$

Actions are ranked by gain, ties broken by the number of open jobs requiring the skill. The what-if simulator applies the same rule to any set of actions; on the Career Map each job is drawn at distance 100 − score from the student, so the 70% ring separates jobs within reach from the rest.

**(d) Skill Roadmap.** Because skills pay off together, the roadmap searches exactly over bundles $B$ of 1, 2 and 3 actions, at most one per skill, and for each size keeps the bundle with the most jobs within reach and then the least effort

$$\text{effort}(B) = \Sigma_{(x,\, t) \in B} \left( t - \ell_s(x) \right) \tag{4.5}$$

A bundle is shown only if it beats every smaller one, and it is marked **works together** when its gain exceeds the sum of the single gains (4.4) of its steps. For every nearby job the roadmap also finds the cheapest subset of that job's own gaps that brings it within reach, or names the CGPA cutoff when no skill can help (Algorithm 1). With about 30 candidate actions a student needs roughly 10 bundle checks.

**Algorithm 1:** Skill Roadmap for one student

```
input: levels L, CGPA g, open jobs J
A ← every (skill, target level) some job asks for above L
for k ← 1 to 3 do
    best ← bundle of k actions, one per skill, with most jobs within reach, then least effort
    if reach(best) > reach of every smaller bundle shown then
        order steps by immediate effect; mark works together if gain > sum of single gains
for each job j not within reach do
    if g < cutoff(j) then report the CGPA blocker
    else cheapest subset of j's gaps that brings j within reach
return bundles, paths to the closest jobs, number of bundles checked
```

**(e) Hidden Talent.** For a recruiter's job the rule is run from the job's side over every student who gave placement consent and has not applied to that job. Students for whom the job is within reach are listed, and so are students for whom one action from (c) would bring it within reach, with that action named. The recruiter can send an invitation; a unique (job, student) constraint prevents repeats.

**(f) Workshop Planner.** Let $S$ be the students, $J$ the open jobs and $C$ the candidate skills. A plan is a set $T \subseteq C$ of $k$ workshops; a workshop raises every attendee to at least Intermediate. With $n_s(T)$ the number of jobs within reach of $s$ after the plan, the objectives are

$$f_{\text{reach}}(T) = |\{\, s : n_s(T) > 0 \,\}|, \qquad f_{\text{match}}(T) = \Sigma_s\, n_s(T) \tag{4.6}$$

The goal *Reach every student* maximises $(f_{\text{reach}}, f_{\text{match}})$ in that order and *Most matches overall* maximises $(f_{\text{match}}, f_{\text{reach}})$. The objective is not submodular: for one student with {Git} and one job requiring {Java, Spring Boot, Git}, f({Java}) = f({Spring Boot}) = 0 while f({Java, Spring Boot}) = 1, so greedy selection can miss the best plan and the $1 - 1/e$ guarantee [5] does not hold. The planner therefore enumerates all $C(|C|, k)$ subsets with bit masks (one per level and weight class), so each test of (4.3) costs a few OR, AND and population-count operations; it falls back to greedy and says so if $C(|C|, k) \cdot |S| \cdot |J|$ would exceed $3 \times 10^8$. Each student is invited only to the smallest subset of the plan that gives them the same result.

**(g) Résumé score, readiness and assessments.** The résumé score out of 100 is computed from ten fixed rules (Table 4.2). Readiness points for the leaderboard add 15 per verified skill, 10 per

verified certificate and 5 per job within reach. An assessment draws 8 questions (4 basic, 4 advanced) for one of 15 skills from a bank of 120, shuffles the options, stores their order on the server and never sends the answers; 8 correct verify Advanced, 6–7 Intermediate, 4–5 Beginner, and below 4 a retake waits 24 hours.

**Table 4.2 Résumé score rules**

| Rule | Points | Maximum |
|---|---|---|
| Résumé uploaded | 15 | 15 |
| About you: 150+ characters (5 points for 50+) | 10 | 10 |
| Skills listed, up to 5 | 4 each | 20 |
| Skills at Intermediate or above, up to 5 | 2 each | 10 |
| Skills verified by assessment or certificate, up to 3 | 5 each | 15 |
| Verified certificates, up to 2 | 5 each | 10 |
| GitHub link / LinkedIn link | 5 / 5 | 10 |
| Introduction video | 5 | 5 |
| CGPA, 10th and 12th marks filled in | 5 | 5 |
| **Total** | | **100** |

**(h) Grounded AI.** The Copilot uses Gemini function calling [16] with eight read-only tools (snapshot, matches, opportunities, simulation, job explanation, application status, and two action proposals). The server runs each tool for the student identified by the token, proposed actions run only when the student clicks them, and every answer lists the data it used. The résumé skill finder extracts text with PDFBox or Apache POI and keeps a Gemini suggestion only if its quoted evidence line occurs in the résumé and the skill is in the catalogue; without a key, word-boundary patterns are used (JavaScript is not Java; “react quickly” is not React).

## 4.5 Database Design

Fig 4.3 shows the entity–relationship diagram. The `users` table holds every account with a role. Students and recruiters each have a one-to-one profile. Skills are a shared catalogue. A student's skills are stored with a level and, separately, the verified level. A job's skills are stored with a minimum level and a must-have or nice-to-have flag. Applications and invitations each link one job and one student, with a unique constraint on the pair. Certificates and assessment attempts belong to a student and a skill. Hibernate creates the 16 tables from the entity classes.

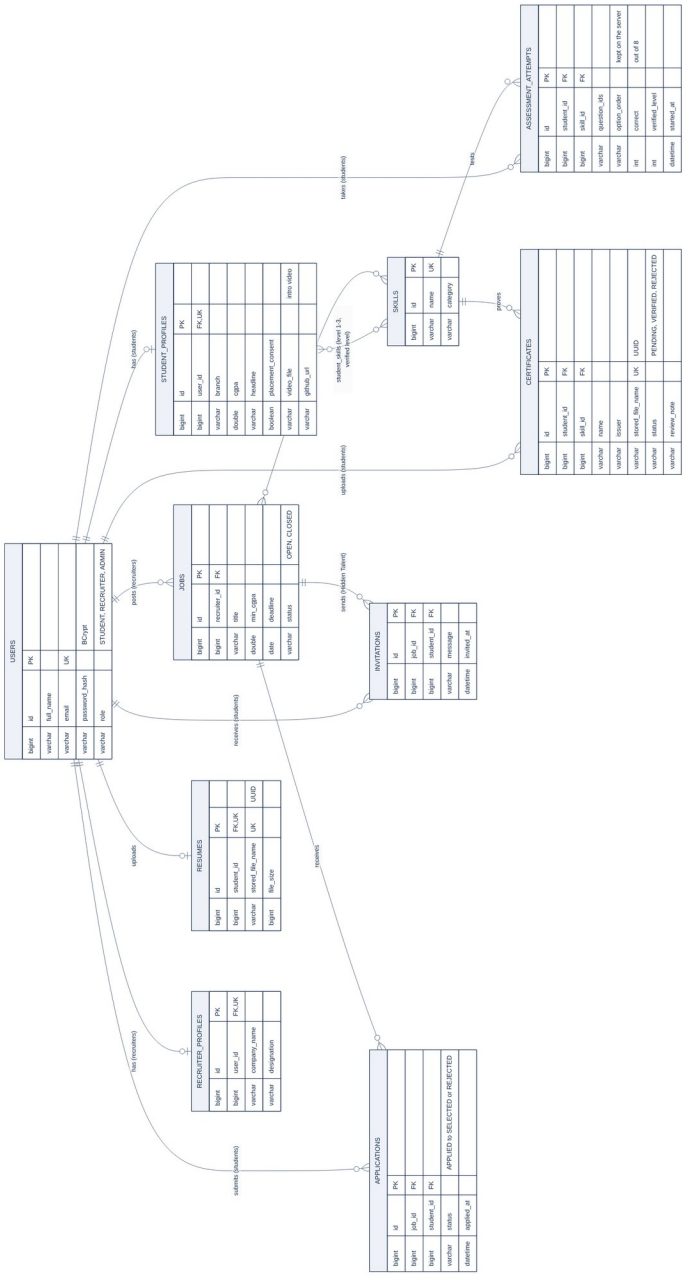



**Fig 4.3 Entity–relationship diagram (rotated to fit the page)**

## 4.6 API Design

The REST API is grouped by role. All endpoints except register, login, health and the skill catalogue need a valid token, and each prefix is open to one role only. Errors return a status code with a plain-language message and, for forms, a message per field. Table 4.3 lists the main endpoints; the Appendix lists all of them.

**Table 4.3 Main REST endpoints**

| Method and path | Role | Purpose |
|---|---|---|
| POST /api/auth/register, /login | Public | Create an account; log in and receive a token |
| GET /api/student/dashboard | Student | Career Map data, top matches, next best action |
| GET /api/student/opportunities | Student | Opportunity Math ranking |
| GET /api/student/what-if?skills= | Student | Re-score all jobs with extra skills or levels |
| GET /api/student/roadmap | Student | Best 1-, 2-, 3-step bundles and paths to jobs |
| POST /api/student/applications | Student | Apply to a job |
| POST /api/student/assessments/start | Student | Start a timed assessment |
| POST /api/student/copilot | Student | Ask the Copilot |
| POST /api/recruiter/jobs | Recruiter | Post a job with skill levels |
| GET /api/recruiter/jobs/{id}/applicants | Recruiter | Ranked, explained applicants |
| GET /api/recruiter/jobs/{id}/talent | Recruiter | Hidden Talent for a job |
| POST /api/recruiter/jobs/{id}/invite/{sid} | Recruiter | Invite a student to apply |
| GET /api/admin/workshops?rounds=&goal= | Admin | Workshop plan |
| PATCH /api/admin/certificates/{id} | Admin | Verify or reject a certificate |

## 4.7 Data Flow Diagrams (Levels 0, 1 and 2)

The movement of data is modelled at three levels of detail in Fig 4.4.

- **Level 0 (context).** CareerSphere is a single process exchanging data with three external entities: students send profiles, résumés, answers and applications and receive scores, plans and the roadmap; recruiters send jobs, status updates and invitations and receive ranked applicants and hidden talent; the placement cell sends reviews and workshop requests and receives analytics and plans.
- **Level 1 (main processes).** P1 authentication, P2 profile and verification, P3 matching and planning, P4 jobs and applications, and P5 the Copilot and résumé finder, with two data stores: D1 the MySQL database and D2 the uploads folder.
- **Level 2 (P3 matching and planning).** P3 is expanded into loading levels and jobs, scoring every job in half units, generating learning actions, searching bundles of one to three actions, and returning the best bundle of each size with paths and CGPA blockers.

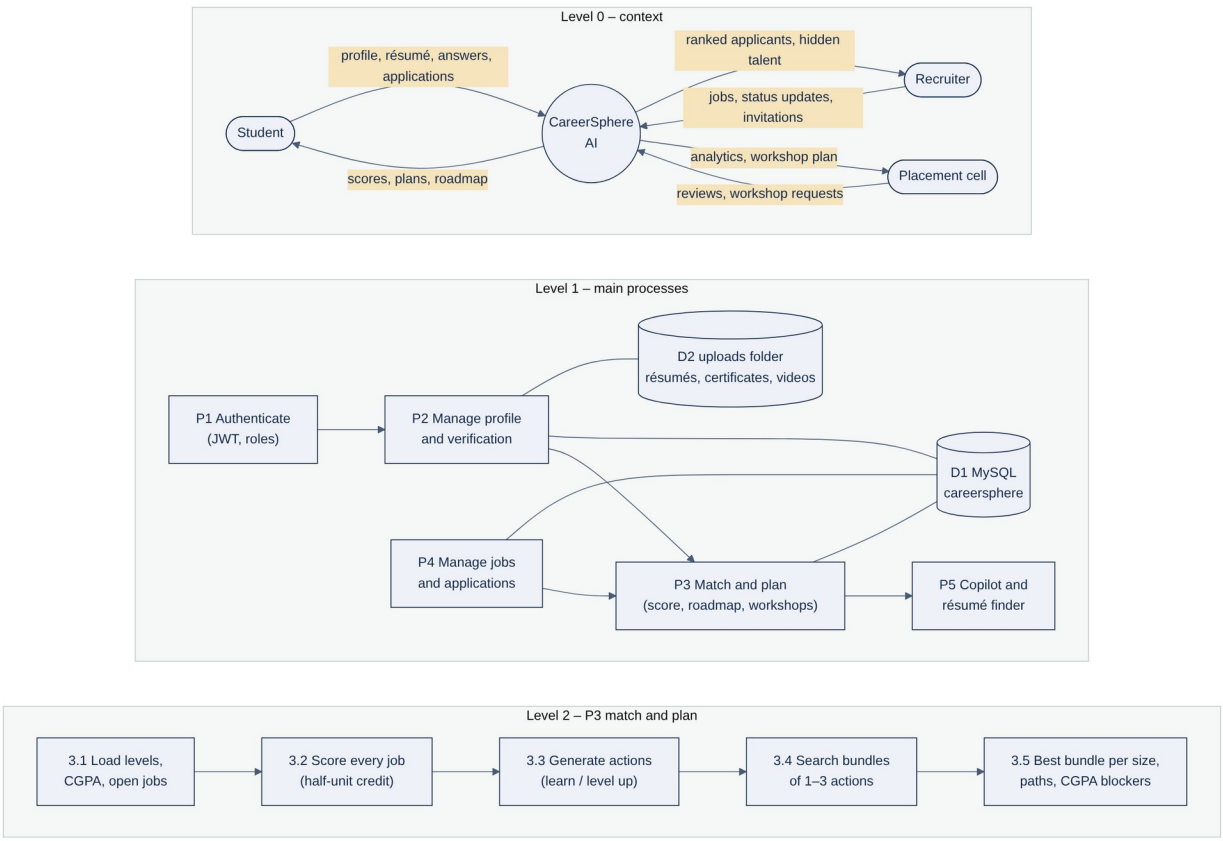



**Fig 4.4 Data flow diagrams at Level 0 (context), Level 1 (processes) and Level 2 (matching and planning)**

## 4.8 UML Design Models

Five kinds of UML diagram document the structure and behaviour of the implementation.

**Use case view.** The actors are the student, the recruiter and the placement cell, with the Gemini service as an external actor used by the Copilot and the résumé finder (Fig 4.5). Use cases in green are the new decision-support features.

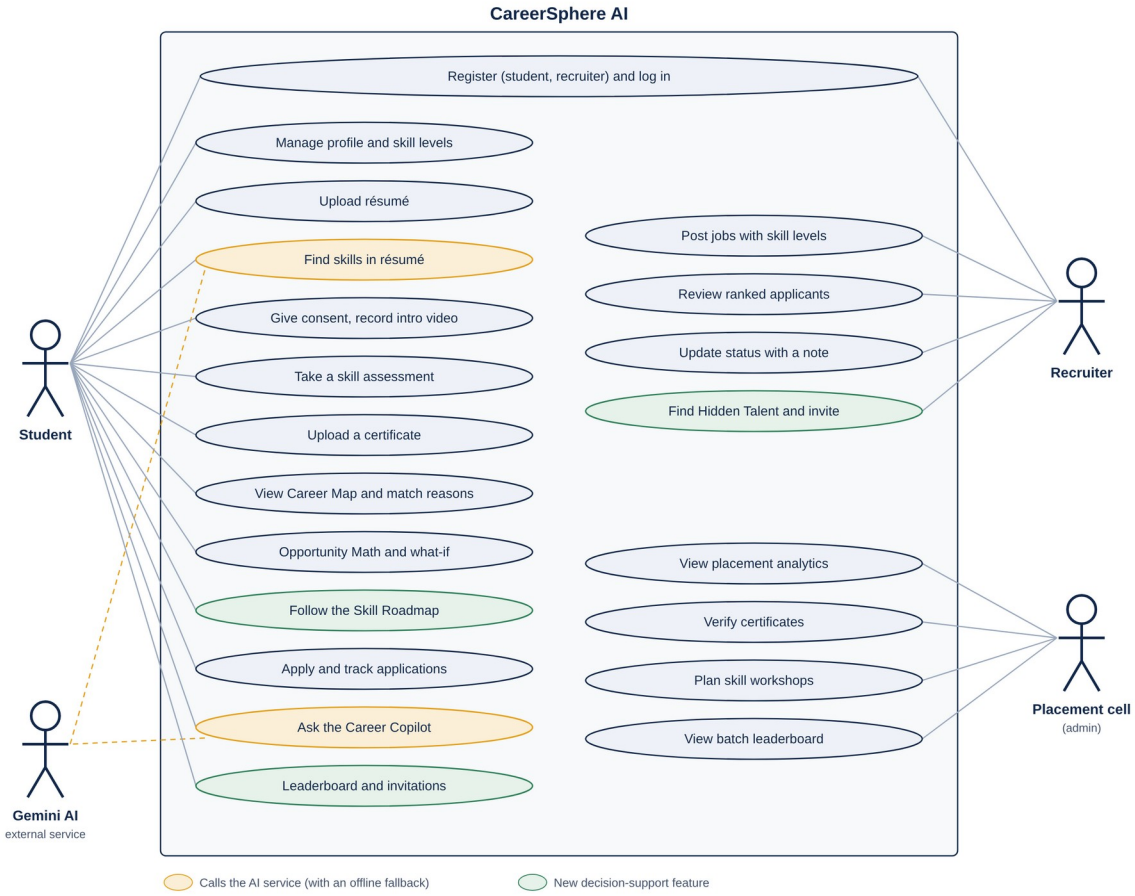



**Fig 4.5 Use case diagram of CareerSphere AI**

**Class view.** The main entities (User, StudentProfile, Job, Skill, Application, Certificate, AssessmentAttempt, Invitation) and the services of the rule engine, which all depend on MatchService, are shown in Fig 4.6.

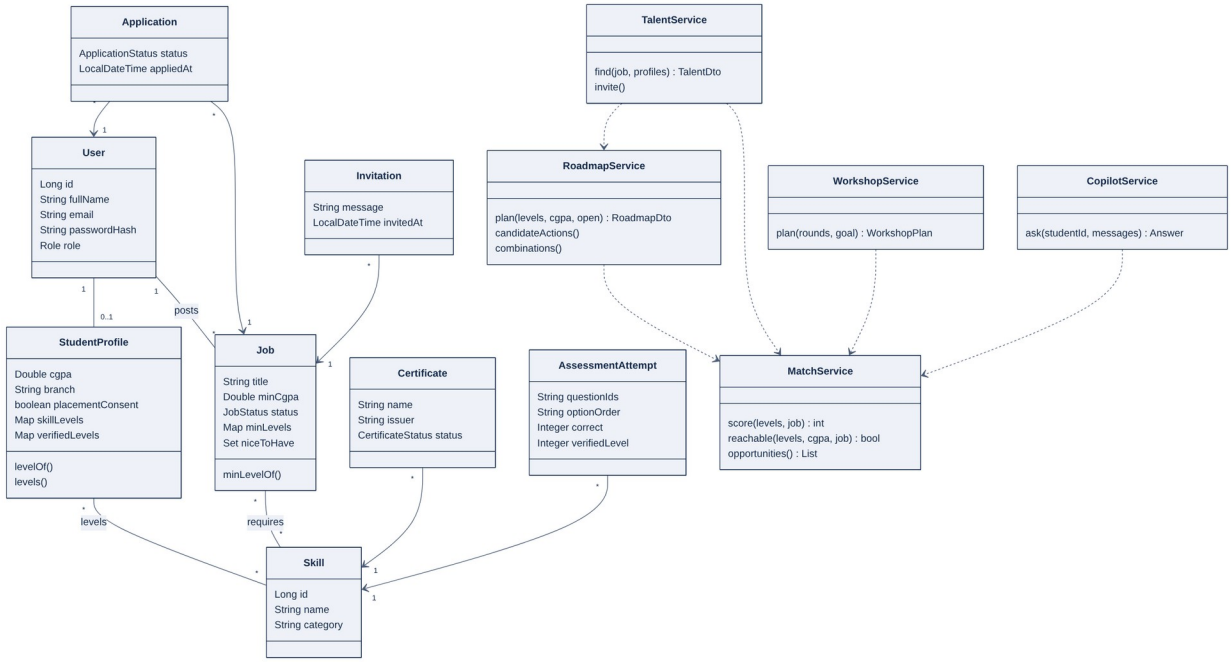



**Fig 4.6 Class diagram of the principal entities and services**

**Sequence view.** Figs 4.7 to 4.11 trace login, applying to a job, asking the Copilot, finding skills in a résumé, and building a Skill Roadmap through the layers.

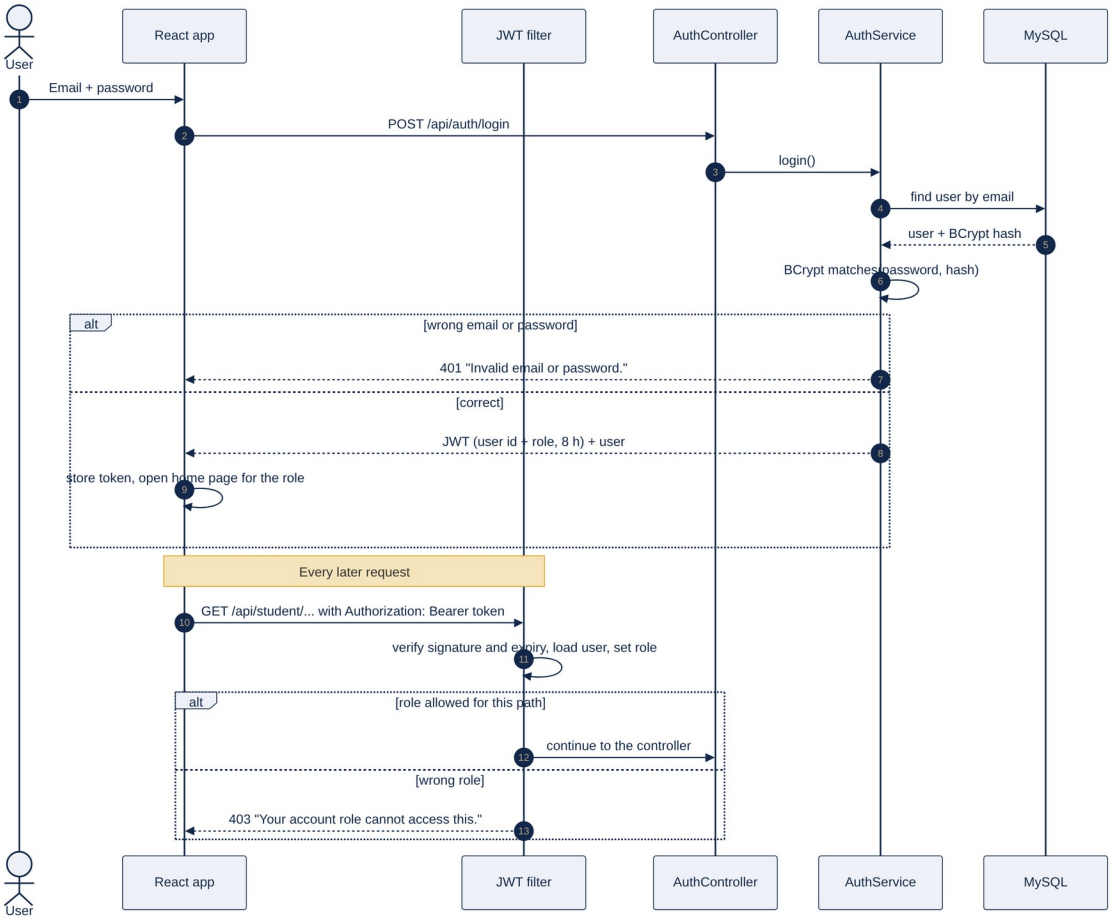



**Fig 4.7 Sequence diagram of login and an authenticated request**

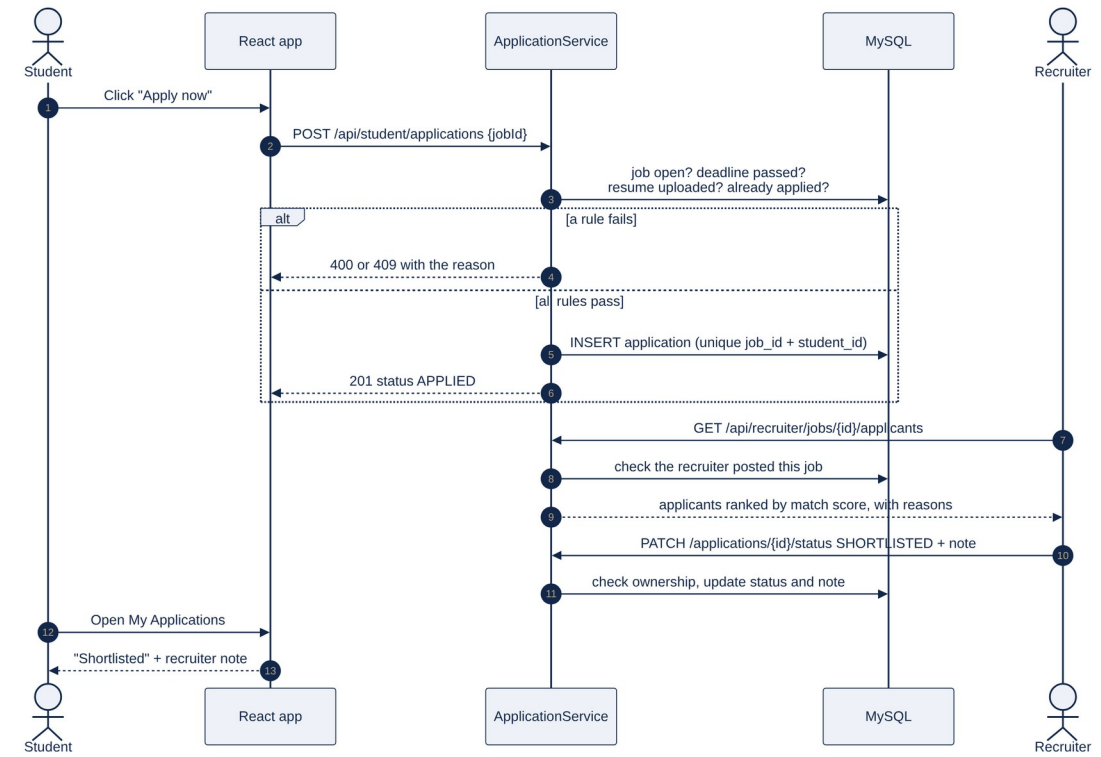



**Fig 4.8 Sequence diagram of applying to a job**

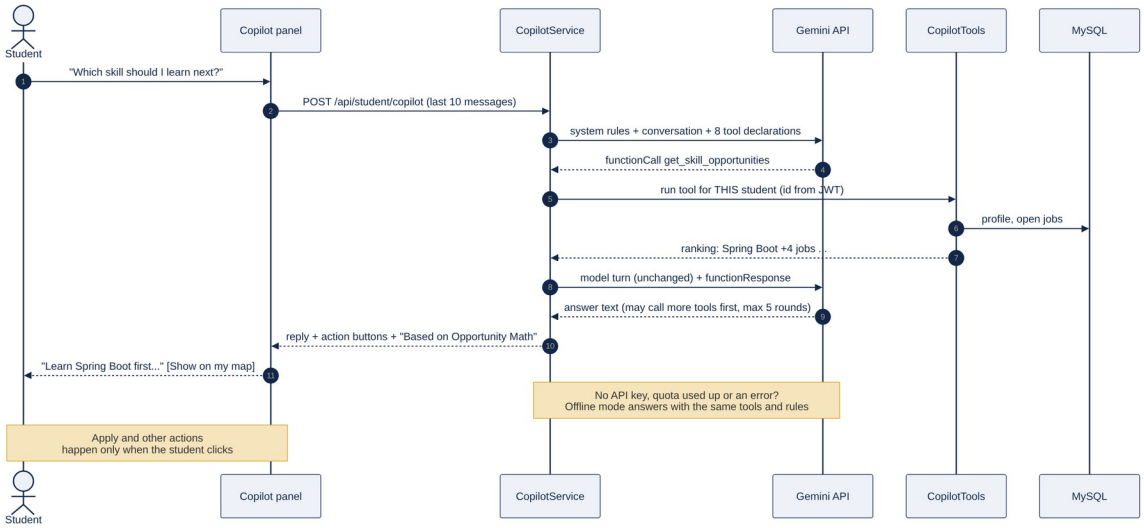



**Fig 4.9 Sequence diagram of the Copilot with function calling**

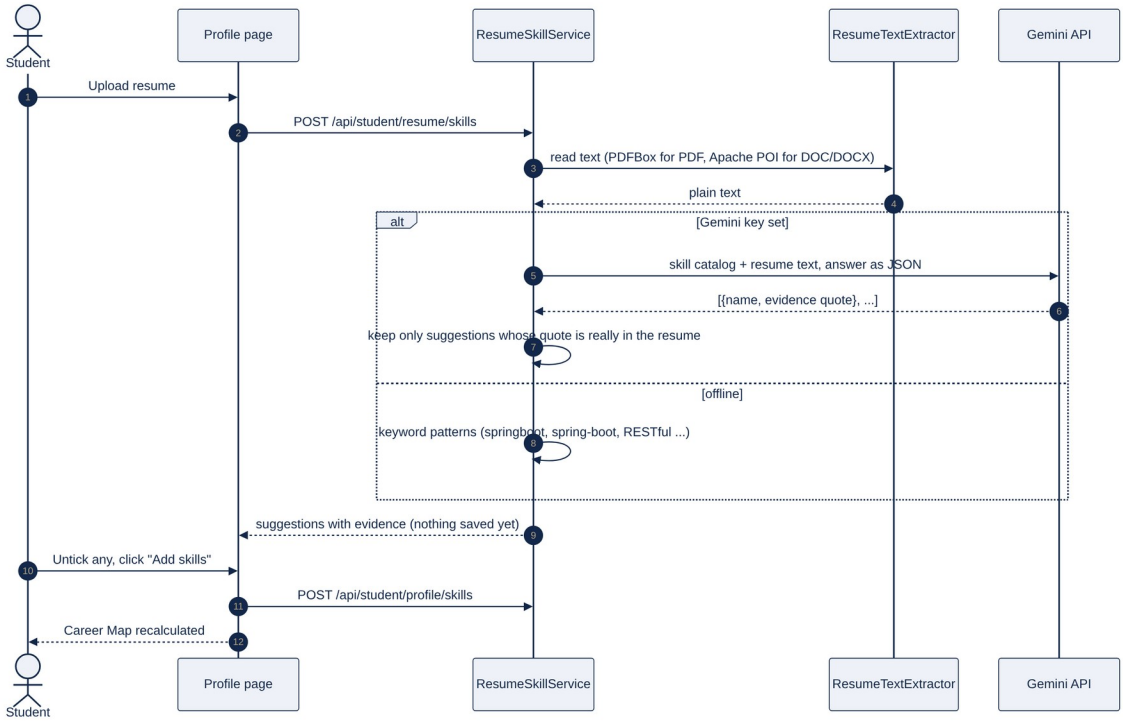



**Fig 4.10 Sequence diagram of the evidence-checked résumé skill finder**

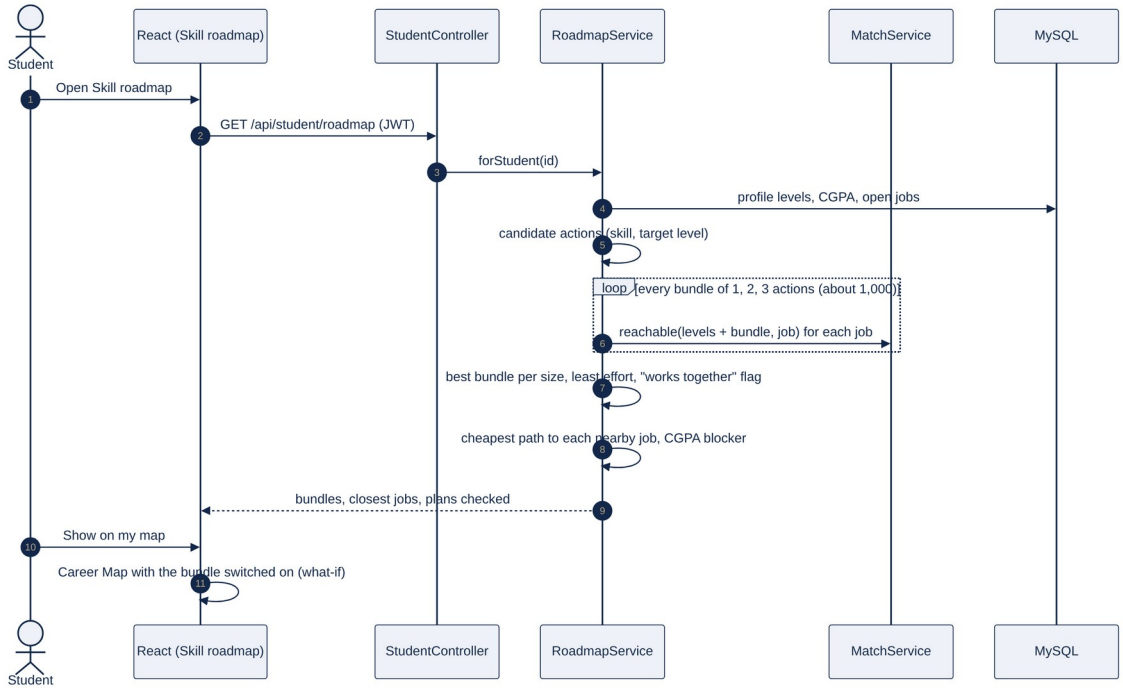



**Fig 4.11 Sequence diagram of building a Skill Roadmap**

**Activity view.** Fig 4.12 shows the student's path from login through résumé, consent and verification to applying, with the loop back through the Skill Roadmap or a workshop when no job is within reach yet.

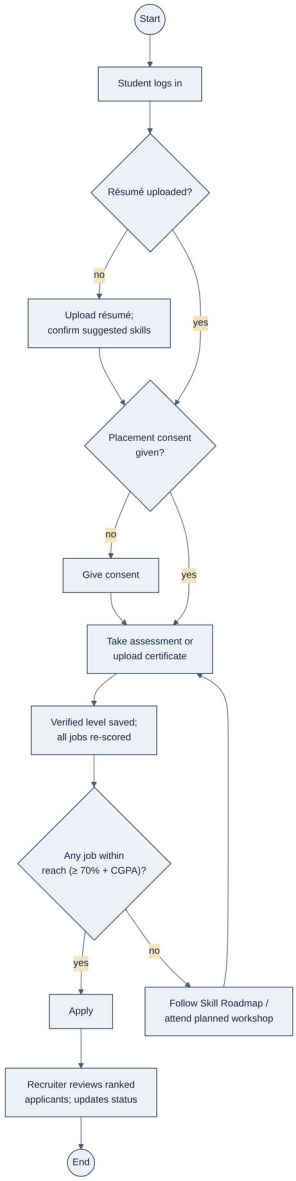



**Fig 4.12 Activity diagram of the student placement flow**

**Deployment view.** Fig 4.13 shows the development set-up on a Mac (browser or Mac app, Spring Boot on port 8080, MySQL on port 3306, the uploads folder), the Android emulator reaching the same web app, the Gemini API called from the server only, and the online deployment as one Docker image with a managed MySQL database.

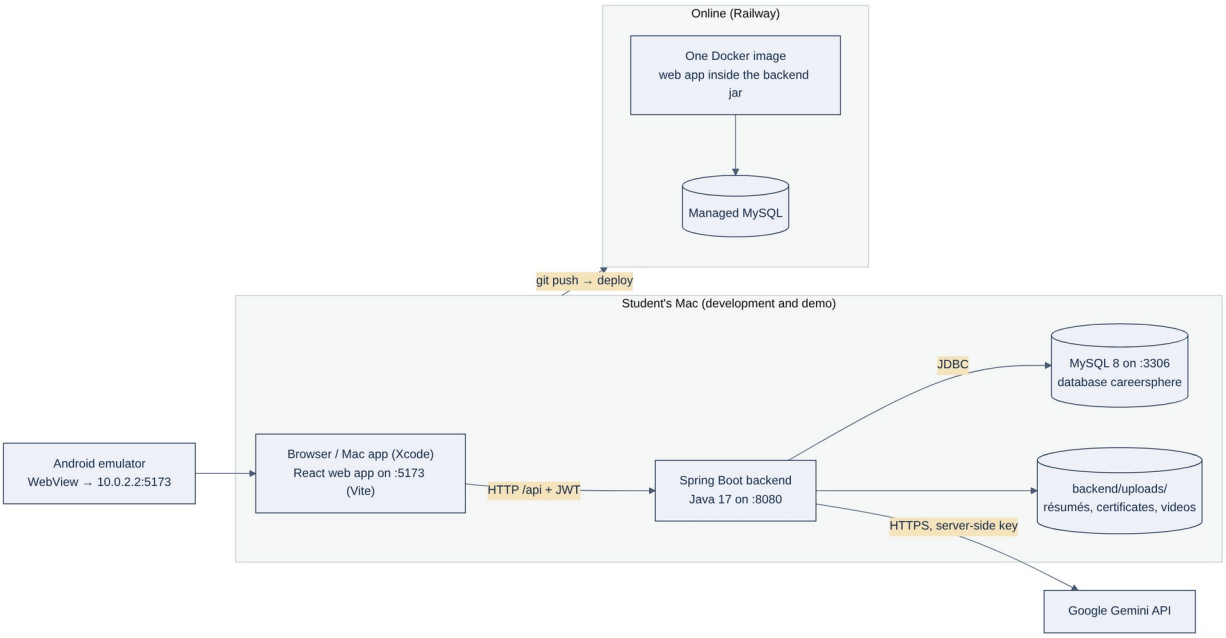



**Fig 4.13 Deployment diagram of the local and online environments**

**Workshop planning flow.** Fig 4.14 summarises the exact Workshop Planner from method (f).

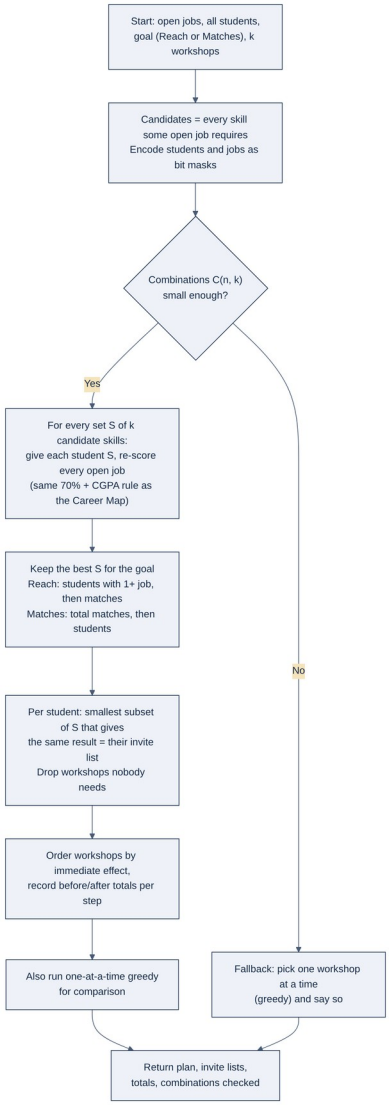



**Fig 4.14 Flow of the exact Workshop Planner**

# CHAPTER-5

# TESTING

## 5.1 Need for Testing and Verification Strategy

Every number CareerSphere shows is used to make a decision: which skill to learn, whom to shortlist, which workshop to run. A wrong rule would therefore mislead students and the placement cell without any visible error. The verification strategy has three levels. **Unit tests** check the rules in isolation on small, hand-checkable cases, so every expected value can be worked out on paper. **API tests** start the whole application on an in-memory H2 database and call the real endpoints through the real security filter chain with MockMvc. **User-interface checks** walk through every screen in a real browser at desktop and phone widths and fail on any script error.

## 5.2 Unit Testing of the Rules

There are 39 unit tests in six classes (Table 5.1), all passing. As a check that the tests can detect a faulty rule, the threshold comparison was changed from ≥ to >; the inclusive-threshold test failed at once, and passed again when the change was reverted.

**Table 5.1 Unit tests**

| Class | Tests | What they check |
|---|---|---|
| MatchServiceTest | 9 | Share of skills; rounding (2 of 3 = 67%); 100% case; CGPA cutoff reported separately; missing CGPA; inclusive 70% threshold; Opportunity Math ranking, CGPA exclusion, known skills never suggested |
| SkillLevelTest | 6 | Defaults equal the plain rule; half credit below the level; nice-to-have weight; level-up suggestions; workshop to Intermediate; what-if "Skill:2" syntax |
| WorkshopServiceTest | 6 | Baseline counts; first-reach ordering; three workshops reach all students; unneeded workshops dropped; no invitation for a known skill; a complementary case where greedy fails |
| RoadmapTalentTest | 4 | Two-step bundle when no single skill helps, marked "works together"; least effort wins ties; cheapest path and CGPA blocker; Hidden Talent excludes non-consented students and applicants |
| PlacementProfileTest | 8 | Résumé score adds up and never passes 100; steps in order; leaderboard privacy; grading levels; lock before consent; answers never sent; retake wait; resuming a running test |
| ResumeSkillServiceTest | 6 | Skills in project lines with evidence; common spellings; JavaScript is not Java; ordinary words not mistaken for skills; known skills marked; long evidence trimmed |

## 5.3 Integration and API Testing

Table 5.2 lists the 17 end-to-end test cases. Each runs against a fresh in-memory database with the demo data, through the same security configuration as production, and is run with `mvn test` from the backend folder.

**Table 5.2 End-to-end API test cases**

| ID | Test case | Expected result |
|---|---|---|
| TC-01 | Register; register again with the same email | Token returned; second attempt rejected (409) |
| TC-02 | Register with role ADMIN | Rejected (403) |
| TC-03 | Log in with a wrong password | 401 with a clear message |
| TC-04 | Call a protected API without or with a bad token | 401 |
| TC-05 | Each role calls the other roles' APIs | 403 for every other role's API |
| TC-06 | Save profile with CGPA 11.5; save a valid profile | 400 with a field error; valid profile saved and reloaded |
| TC-07 | Upload a real PDF; upload other files (e.g. an .exe) | PDF accepted; other files rejected (400) |
| TC-08 | Run the résumé skill finder; confirm some skills | Suggestions with evidence; only confirmed skills added |
| TC-09 | Apply without résumé, without consent, then twice | Blocked (400), blocked (400), accepted once, then 409 |
| TC-10 | Another recruiter updates an application | 403; the owner's update is visible to the student |
| TC-11 | What-if with extra skills | Jobs within reach never decrease |
| TC-12 | Ask the Copilot without an AI key | Offline answer built from the student's data |
| TC-13 | Admin requests a workshop plan; a student requests it | Plan returned; student gets 403 |
| TC-14 | Post a job with levels and nice-to-have skills | Saved and used in the score |
| TC-15 | Consent without résumé; assessment before and after consent | Consent blocked; test locked, then graded on the server |
| TC-16 | Upload a certificate; admin verifies it | Skill added to the profile as verified |
| TC-17 | Student roadmap; recruiter talent and invite | Roadmap returned; candidate listed; invitation shown to the student; repeat invite 409 |

## 5.4 User Interface Testing and Validation

A scripted browser (Playwright with Chromium) logs in as each role and opens every screen in turn, including taking and submitting an assessment, sending a Hidden Talent invitation and checking that it appears on the student's Jobs page, and planning workshops. The run fails on any JavaScript error. The same walk is repeated at a phone width of 390 pixels, where it also confirms that no page scrolls sideways. Every form is validated on the server as well as in the browser: invalid input returns status 400 with a message per field, business-rule violations return a plain sentence, and unexpected errors return a generic message so that no stack trace reaches the browser.

# CHAPTER-6

# CODE AND IMPLEMENTATION

## 6.1 Technology Stack and Project Structure

**Table 6.1 Technology stack**

| Layer | Technology | Use in the project |
|---|---|---|
| Frontend | React 18, React Router 6, Vite 5, Axios, SVG | All screens, routing by role, Career Map drawing, API calls with the token |
| Backend | Java 17, Spring Boot 3.3, Spring Web | REST API: controller → service → repository |
| Security | Spring Security, jjwt, BCrypt | Stateless login token; role rules; hashed passwords |
| Data | Spring Data JPA, Hibernate, MySQL 8; H2 | Persistent data; in-memory database for tests |
| AI | Google Gemini API, Apache PDFBox, Apache POI | Copilot and résumé skill finder; résumé text extraction |
| Clients | SwiftUI (Xcode), Kotlin (Android Studio), VS Code | Mac, iPhone and Android apps; tasks and debugging |
| Testing | JUnit 5, MockMvc, Playwright | Unit, API and browser tests |
| Deployment | Docker, Docker Compose, Railway | One image with the web app inside the backend jar |

Fig 6.1 is a mind map of the whole project: what each user can do, the algorithms, and every technology, with where each lives in the code. The backend has 68 Java source files (about 6,400 lines) and 8 test files (about 1,270 lines); the frontend has 43 JavaScript and JSX files (about 3,900 lines); the Android app is about 270 lines of Kotlin and the Mac and iPhone app about 300 lines of Swift.

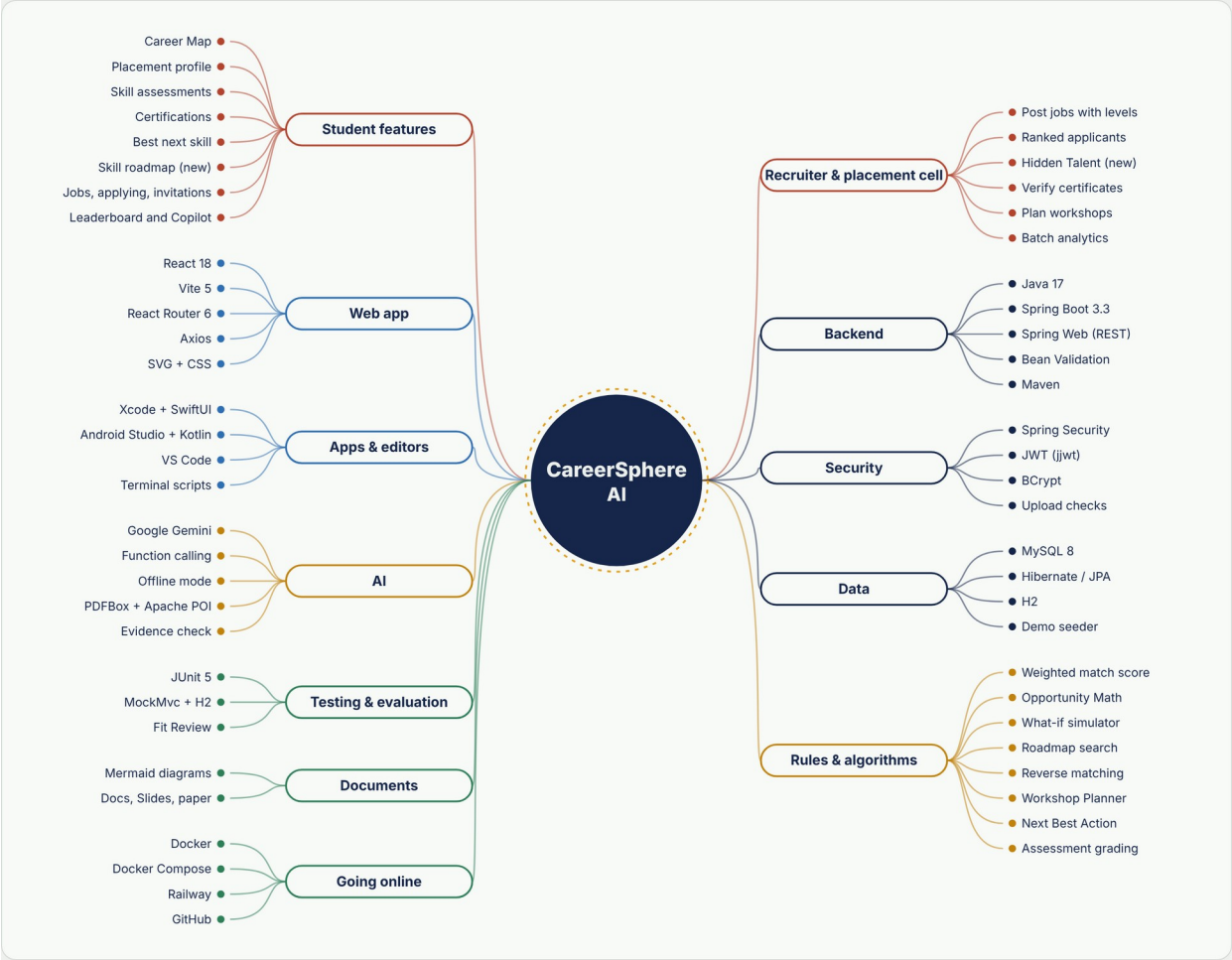



**Fig 6.1 Mind map of the project: features, algorithms and technologies**

**Listing 6.1 Folder structure of the project**

```
careersphere/
├── backend/          Spring Boot API (Java 17, Maven)
│   └── src/main/java/com/careersphere/
│       ├── ai/         Copilot, Gemini client, résumé skill finder
│       ├── config/     security, uploads, demo data seeder
│       ├── controller/ REST endpoints per role
│       ├── entity/     JPA entities        ├── repository/ data access
│       └── service/    match rule, roadmap, talent, workshops, ...
├── frontend/         React + Vite web app (pages/, components/, services/)
├── android/          Android Studio project (Kotlin WebView)
├── CareerSphere.xcodeproj, app/   Xcode project (SwiftUI)
├── .vscode/          tasks and debug configurations
├── docs/             diagrams, report, evaluation kit
└── run.sh, stop.sh, Dockerfile, docker-compose.yml, deploy/
```

## 6.2 Core Source Code

The listings below are the central parts of the rule engine, taken from the project source. Listing 6.2 is the match score, Listing 6.3 the bundle search of the Skill Roadmap, Listing 6.4 the Hidden Talent search, Listing 6.5 the exact workshop search and Listing 6.6 the assessment grading rule.

**Listing 6.2 Match score (service/MatchService.java)**

```
public static int weight(Job job, Skill skill) {
    return job.isNiceToHave(skill) ? 1 : 2;
}
/** Credit in half units: 2w if level >= needed, w if 0 < level < needed, 0 if missing. */
public static int halfCredit(int level, int needed, int weight) {
    if (level <= 0) return 0;
    return level >= needed ? 2 * weight : weight;
}
/** round(100 x half / total), half up, in integers. */
public static int percent(long half, long total) {
    if (total <= 0) return 100;
    return (int) ((200L * half + total) / (2L * total));
}
public int score(Map<Long, Integer> levels, Job job) {
    long half = 0, total = 0;
    for (Skill s : job.getSkills()) {
        int w = weight(job, s);
        total += 2L * w;
        half += halfCredit(levels.getOrDefault(s.getId(), 0), job.minLevelOf(s), w);
    }
    return percent(half, total);
}
public boolean reachable(Map<Long, Integer> levels, Double cgpa, Job job) {
    return cgpaEligible(cgpa, job) && score(levels, job) >= MATCH_THRESHOLD;  // 70
}
```

**Listing 6.3 Bundle search of the Skill Roadmap (service/RoadmapService.java)**

```
for (int k = 1; k <= Math.min(MAX_STEPS, distinctSkills(actions)); k++) {
    List<Action> best = null;
    int bestReach = -1, bestEffort = Integer.MAX_VALUE;
    for (List<Action> combo : combinations(actions, k)) {   // at most one action per skill
        checked++;
        int r = reach(apply(levels, combo), cgpa, open);
        int effort = combo.stream().mapToInt(Action::effort).sum();
        if (r > bestReach || (r == bestReach && effort < bestEffort)) {
            best = combo; bestReach = r; bestEffort = effort;
        }
    }
    // a size can gain nothing while the next size gains a lot: keep looking
    if (best == null || bestReach <= lastReach) continue;
    bundles.add(describe(levels, cgpa, open, before, now, best, single));
    lastReach = bestReach;
}
```

**Listing 6.4 Hidden Talent search (service/TalentService.java, simplified)**

```
for (StudentProfile p : all) {
    if (!p.isPlacementConsent()) continue;                       // consent first
    Long sid = p.getUser().getId();
    if (applications.existsByJobIdAndStudentId(job.getId(), sid)) continue;
    Map<Long, Integer> levels = p.levels();
    int score = match.score(levels, job);
    if (!match.cgpaEligible(p.getCgpa(), job)) continue;
    if (score >= MatchService.MATCH_THRESHOLD) { reach.add(candidate(p, score, null)); continue; }
    for (RoadmapService.Action a : RoadmapService.candidateActions(levels, List.of(job))) {
        if (match.reachable(RoadmapService.apply(levels, List.of(a)), p.getCgpa(), job)) {
            near.add(candidate(p, score, a.label()));             // one step away
            break;
        }
    }
}
```

**Listing 6.5 Exact workshop search with bit masks (service/WorkshopService.java)**

```
private void search(Batch b, Goal goal, int depth, int from, int size, int[] current,
                    long[][] masks, int[] best, int[] bestScore) {
    if (depth == size) {
        int reach = 0, matches = 0;
        for (int s = 0; s < b.students(); s++) {
            int r = b.reach(s, masks[depth]);                    // bit-mask test per job
            if (r > 0) reach++;
            matches += r;
        }
        int primary = goal == Goal.REACH ? reach : matches;
        int secondary = goal == Goal.REACH ? matches : reach;
        if (primary > bestScore[0] || (primary == bestScore[0] && secondary > bestScore[1])) {
            bestScore[0] = primary; bestScore[1] = secondary;
            System.arraycopy(current, 0, best, 0, size);
        }
        return;
    }
    for (int i = from; i <= b.skills() - (size - depth); i++) {
        current[depth] = i;
        System.arraycopy(masks[depth], 0, masks[depth + 1], 0, b.words);
        masks[depth + 1][i >>> 6] |= 1L << (i & 63);
        search(b, goal, depth + 1, i + 1, size, current, masks, best, bestScore);
    }
}
```

**Listing 6.6 Assessment grading (service/AssessmentService.java)**

```
public static int levelFor(int correct) {
    if (correct >= 8) return 3;   // Advanced
    if (correct >= 6) return 2;   // Intermediate
    if (correct >= 4) return 1;   // Beginner
    return 0;                     // nothing verified; retake after 24 hours
}
```

## 6.3 System Execution Logs and Output Verification

The following outputs were produced by running the project code on the demo batch. Listing 6.7 is the unit-test run for the match rule, the Skill Roadmap and Hidden Talent. Listing 6.8 is the output of a verification program that loads the demo data through the real DataSeeder and calls the real services; its numbers are the ones reported in Chapter 7.

**Listing 6.7 Unit-test run (excerpt)**

```
MatchServiceTest
  PASS  Score is the share of required skills the student has (3 of 5 = 60%)
  PASS  Scores are rounded to the nearest whole percent (2 of 3 = 67%)
  PASS  Having every required skill gives 100% and says so
  PASS  CGPA below the cutoff is reported separately and blocks 'within reach'
  PASS  The 70% threshold is inclusive (7 of 10 is within reach, 6 of 10 is not)
  PASS  Opportunity Math ranks the skill that brings the most jobs within reach first
  ...
RoadmapTalentTest
  PASS  Roadmap finds a 2-step bundle when no single skill helps, and marks it as working together
  PASS  Roadmap prefers the bundle with less effort when two unlock the same jobs
  PASS  Path to a job: the cheapest steps for each nearby job, and CGPA named as the blocker
  PASS  Hidden Talent shows only consented students who haven't applied, within reach or one step away
...
39 passed, 0 failed
```

**Listing 6.8 Verification run on the demo batch**

```
k=3 REACH   EXACT now 26/44 after 35/138 skills [Spring Boot, Java, Git] greedy 32/76
k=3 MATCHES EXACT now 26/44 after 35/138 skills [Spring Boot, Git, Java] greedy 29/100
Opportunity Math top pick: mean 1.43 jobs over 35 students, 7 unlock nothing
RM Aarav Sharma now 3 +4 +5 +6 checked 575 [1:[Learn Spring Boot], 2:[Learn Spring Boot,
   Learn Excel], 3:[Learn Spring Boot, Learn Excel, Learn React]]
RM Rohan Gupta  now 2 +0 +3 +6 checked 833 [2*:[Learn Java, Learn SQL],
   3*:[Learn Java, Learn SQL, Learn Spring Boot]]          (* = works together)
Roadmap: 35 students, mean new jobs 1-step 1.43, 2-step 2.89, 3-step 4.71;
   7 have no 1-step gain, 7 of them gain with 2 steps; 59 synergy bundles;
   mean bundles checked 1083
TALENT total within 10 near 9 applications 14
```

## 6.4 Screens of the Application

Figs 6.2 to 6.22 show the main screens, taken from the running application with the demo data.

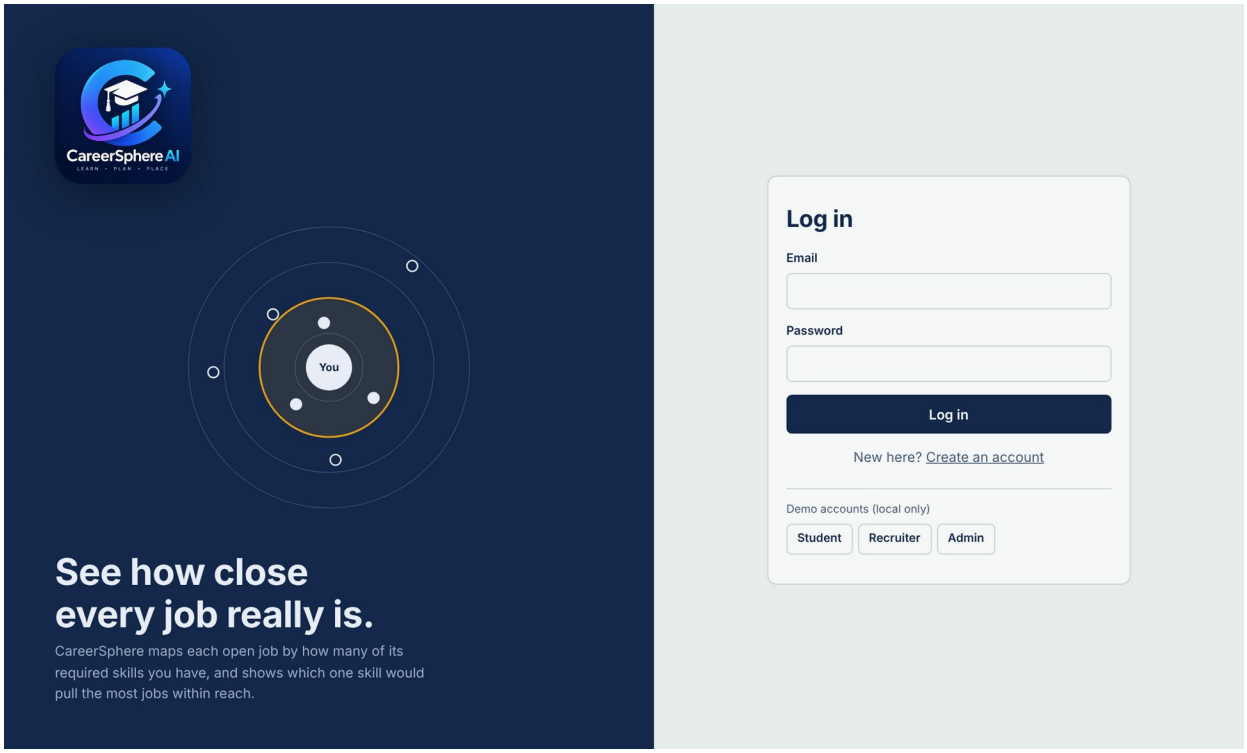



**Fig 6.2 Login page with demo accounts for the three roles**

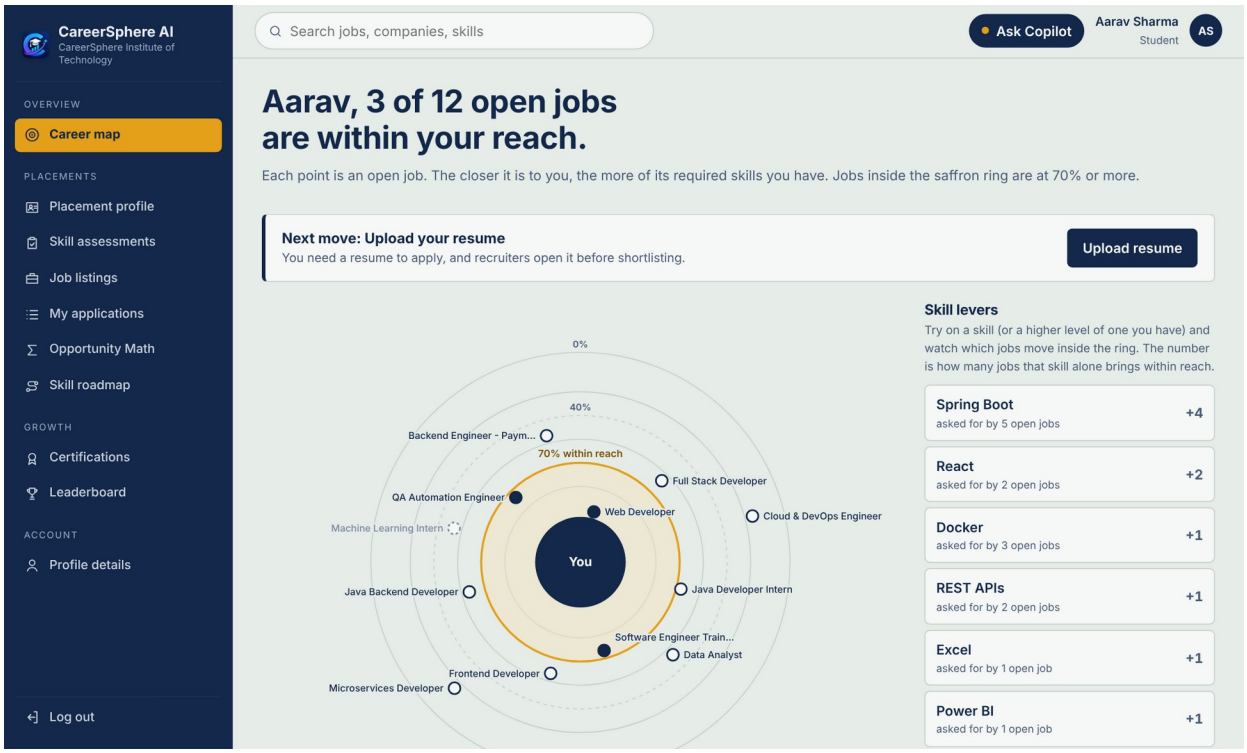



**Fig 6.3 Career Map: each open job placed by match score; skill levers on the right**

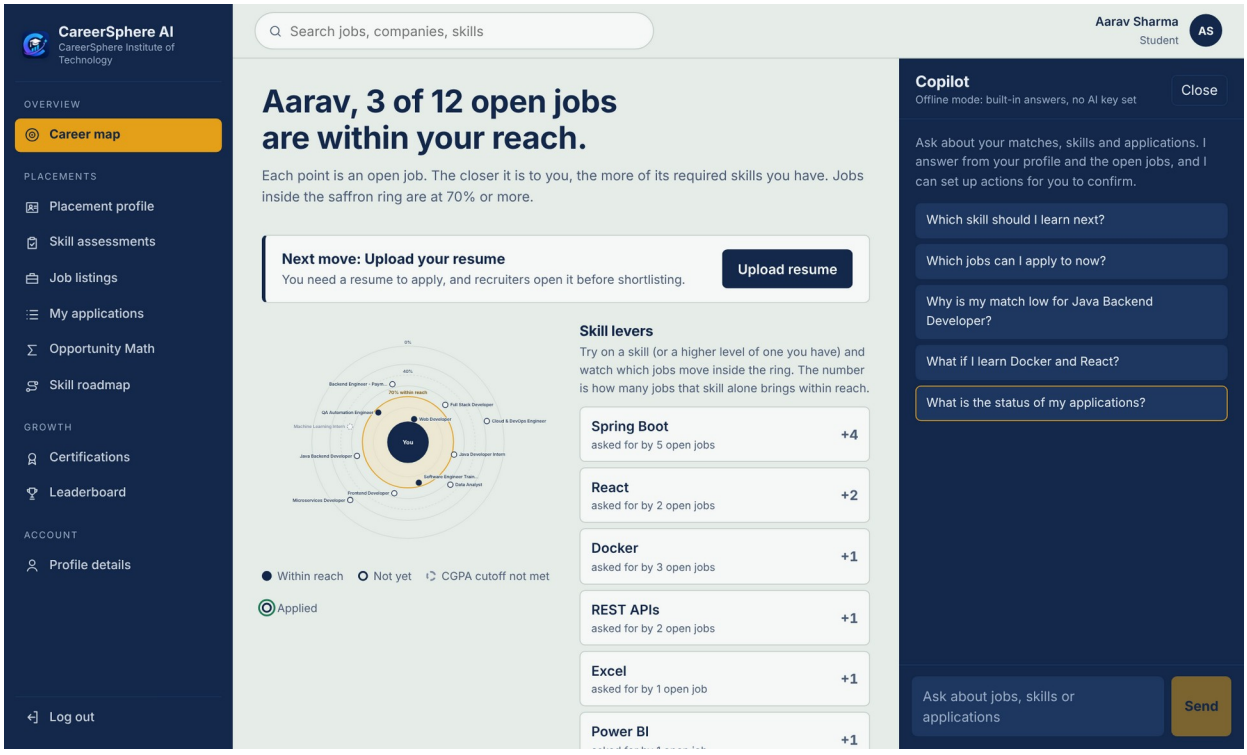



**Fig 6.4 Career Map with the Copilot panel open**

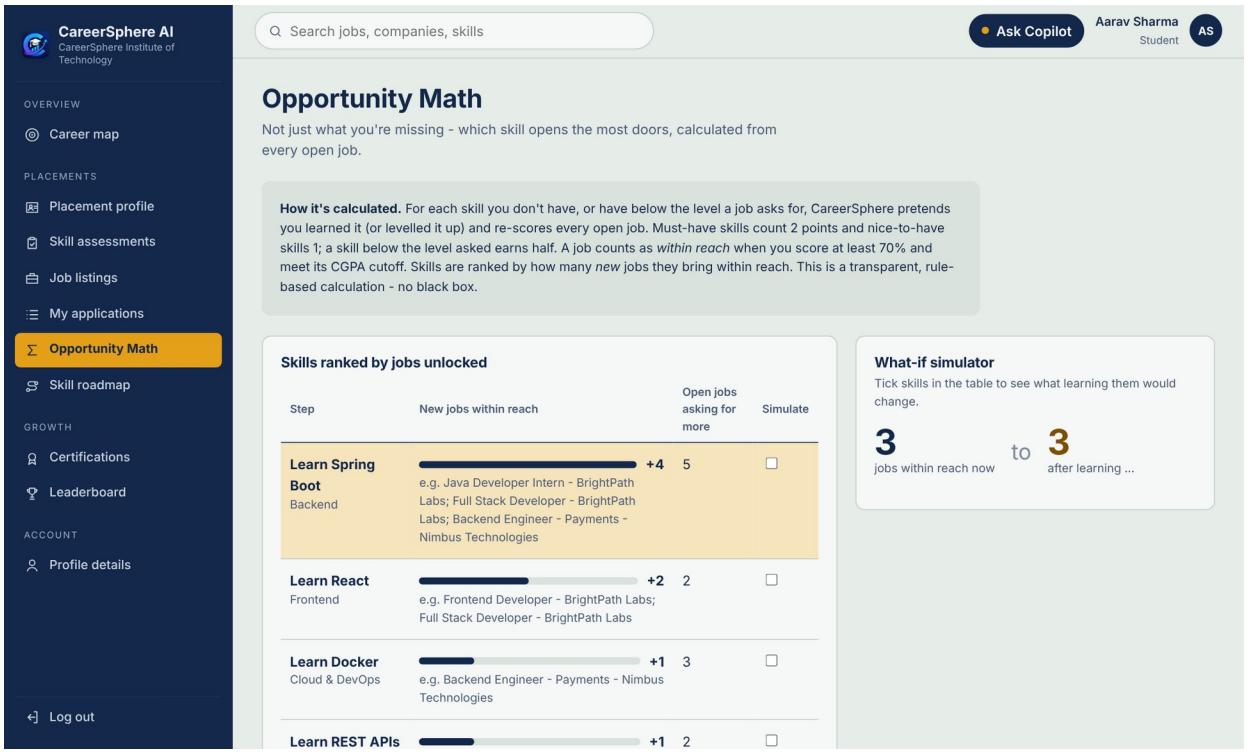



**Fig 6.5 Opportunity Math: the skills that bring the most jobs within reach**

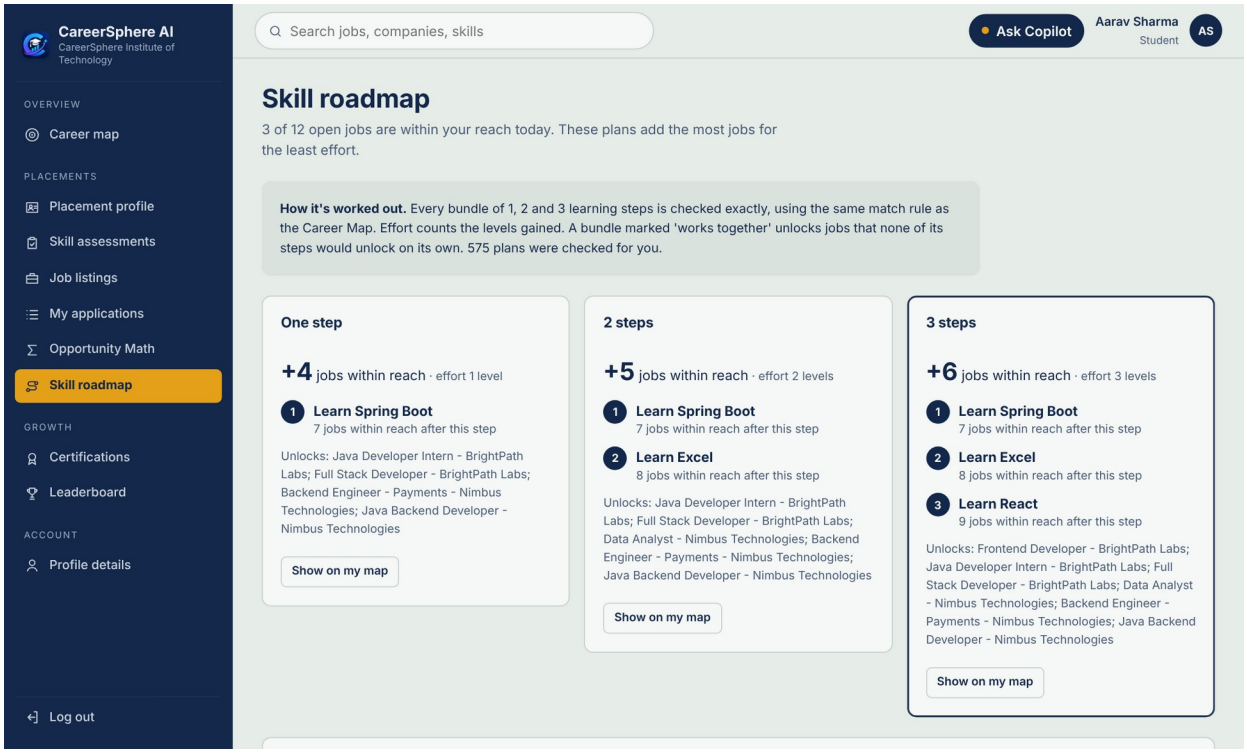



**Fig 6.6 Skill Roadmap: best one-, two- and three-step plans**

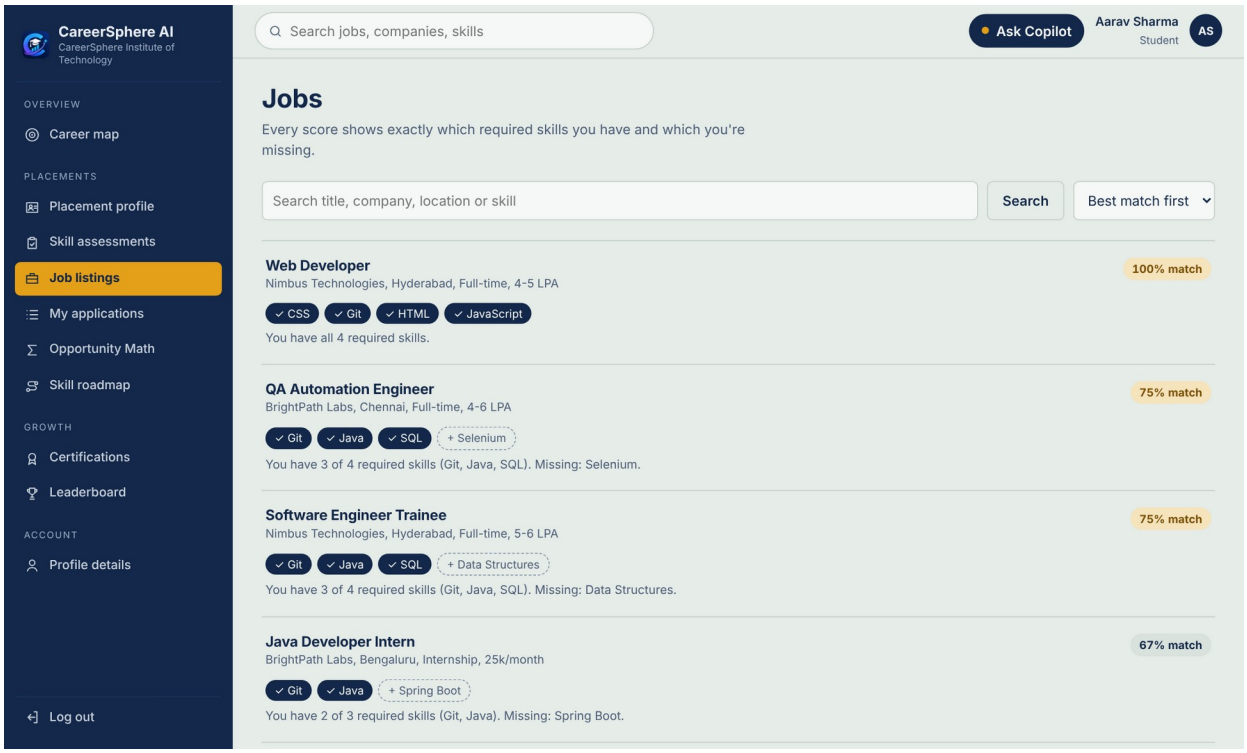



**Fig 6.7 Job listings with explained match scores**

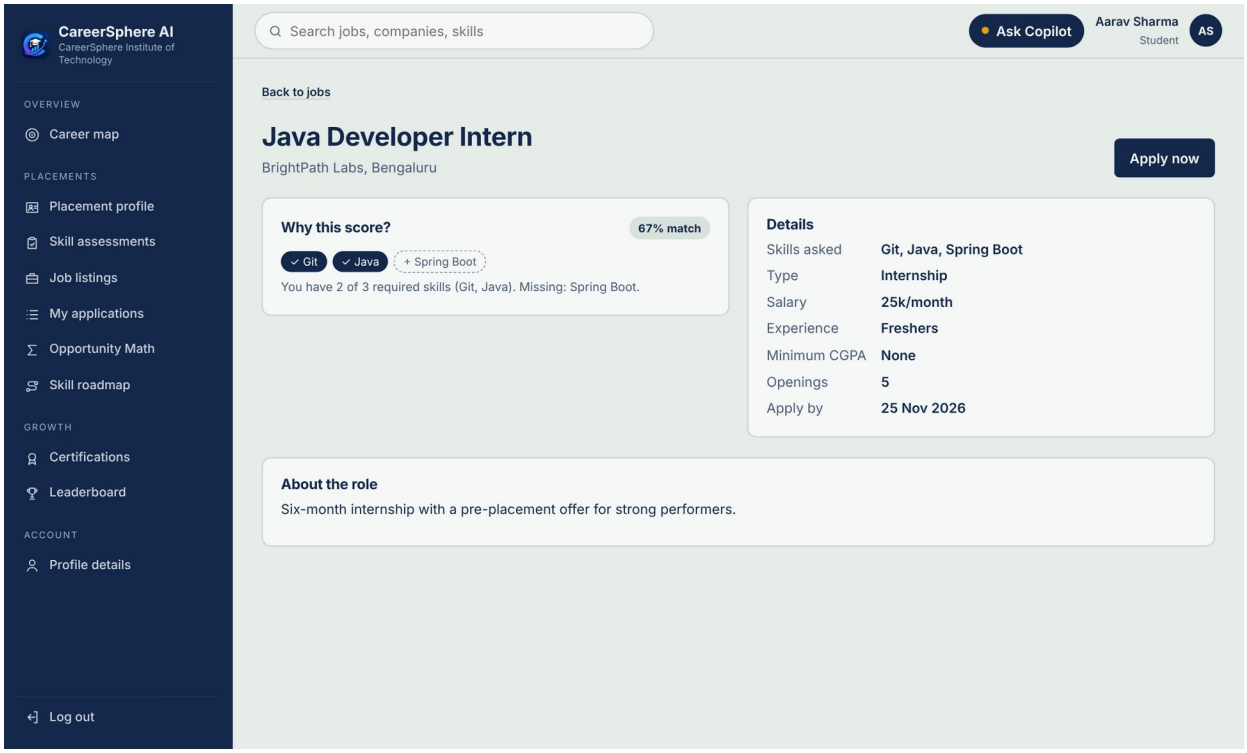



**Fig 6.8 Job detail: why this score, and the job's requirements**

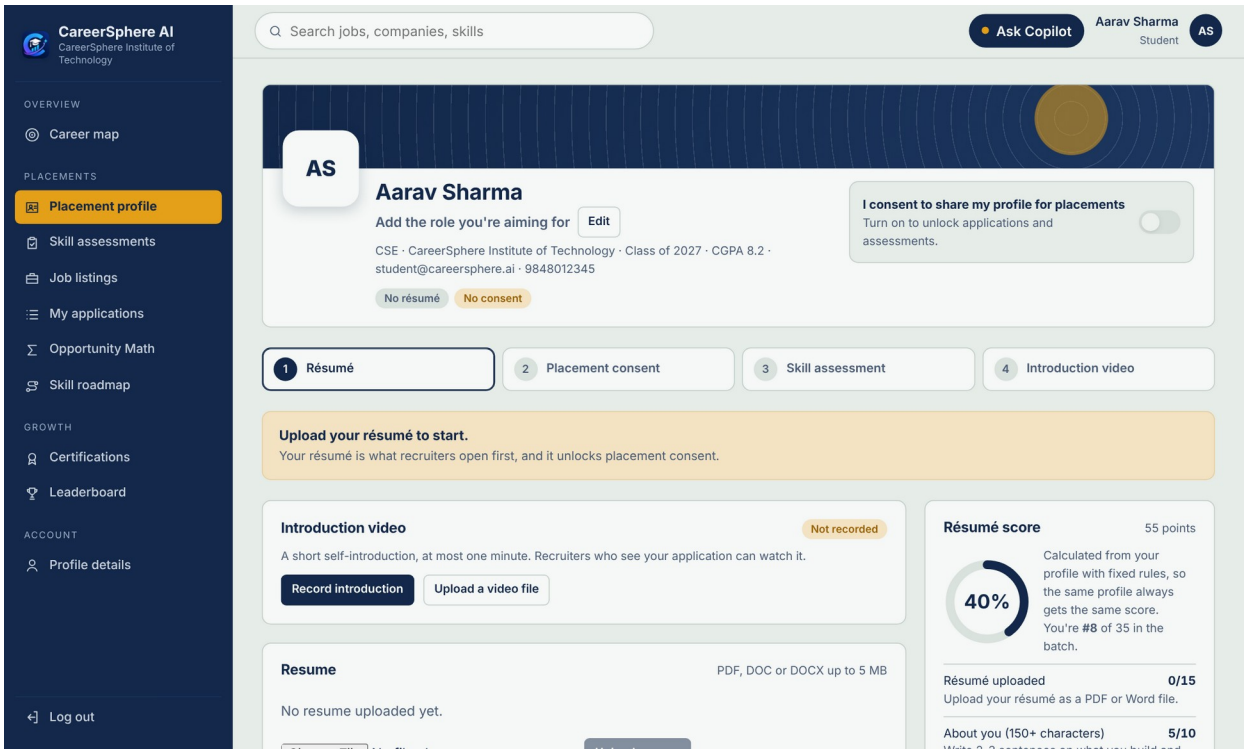



**Fig 6.9 Placement profile of a new student: guided steps and résumé score**

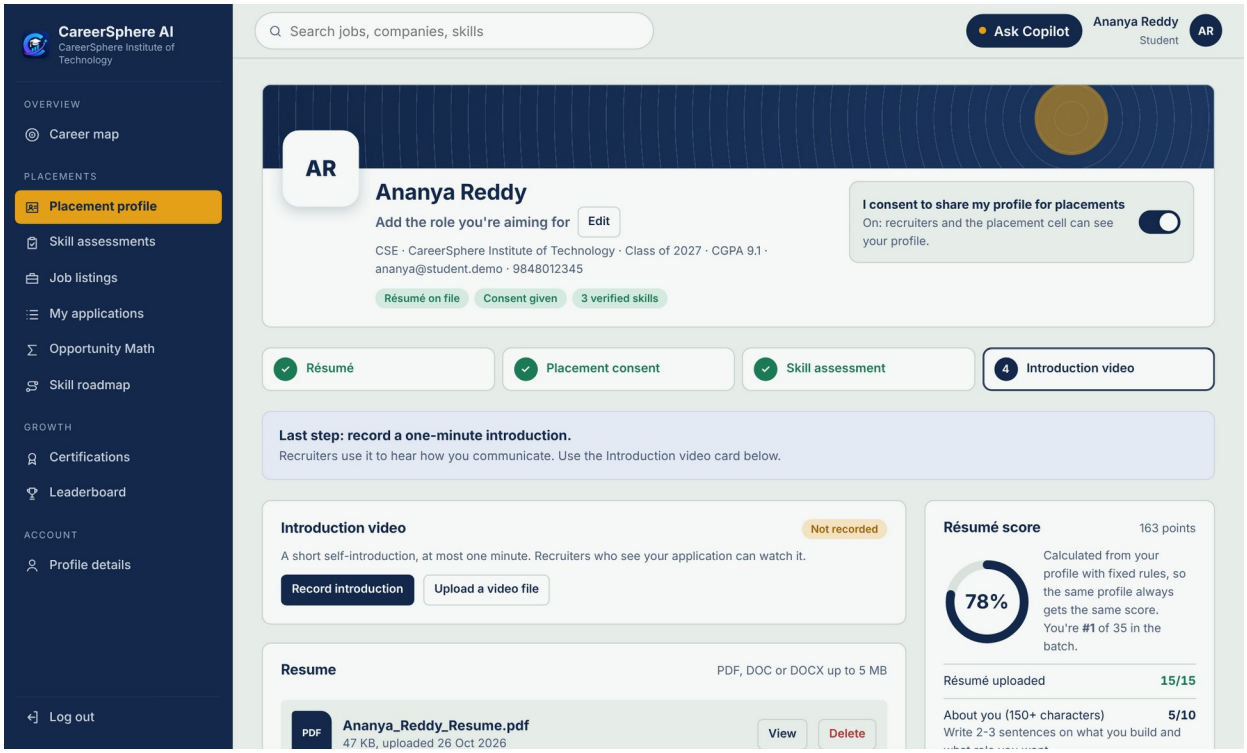



**Fig 6.10 Placement profile of a student who has completed most steps**

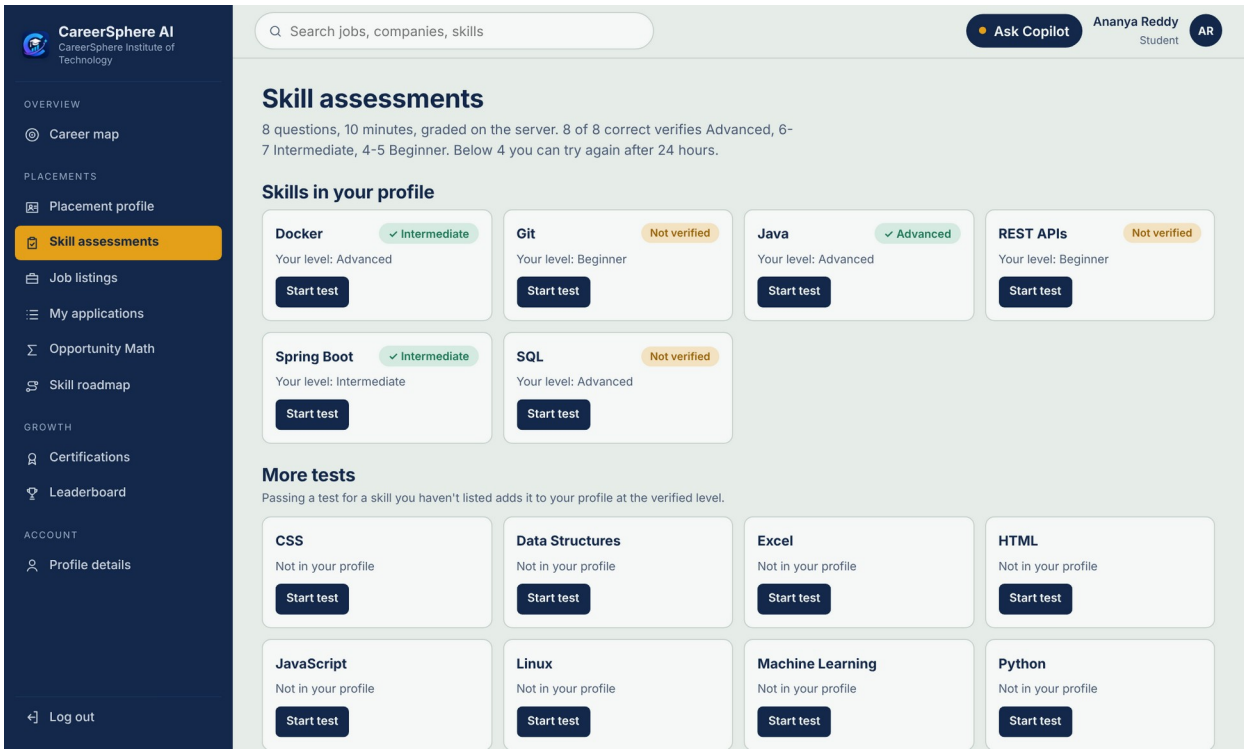



**Fig 6.11 Skill assessments: verified and unverified skills**

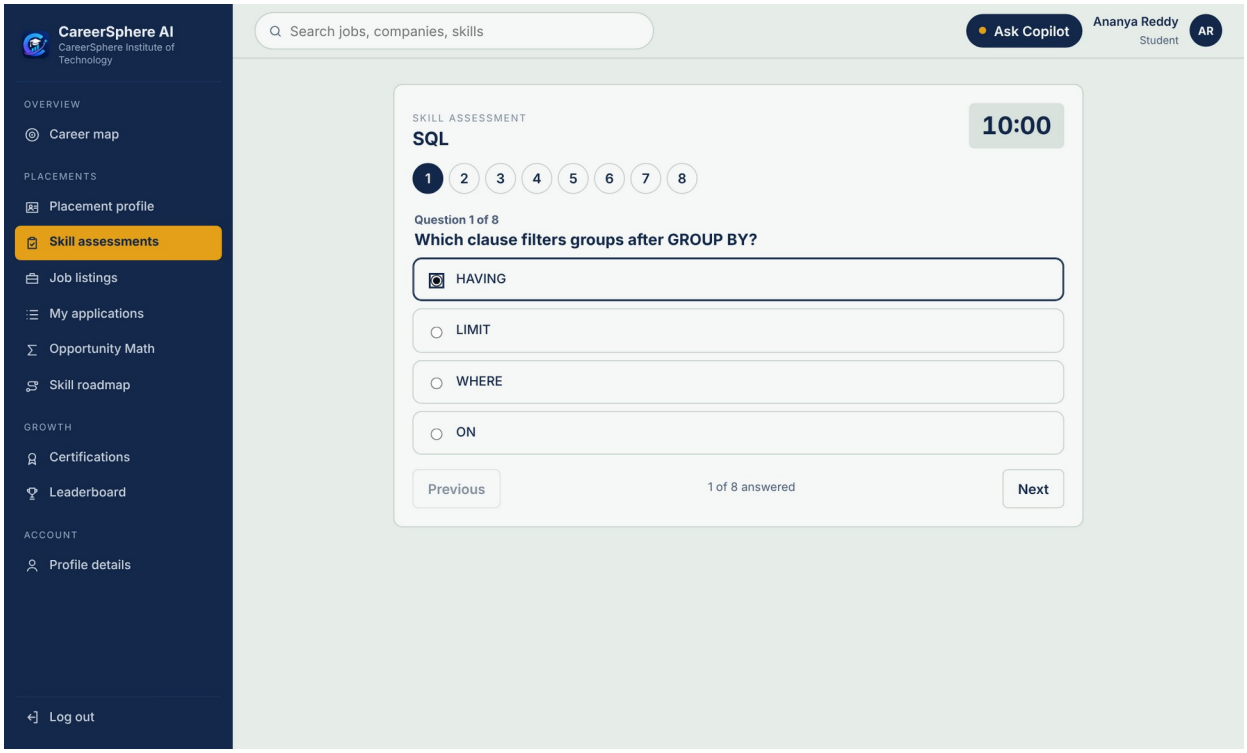



**Fig 6.12 A timed assessment in progress**

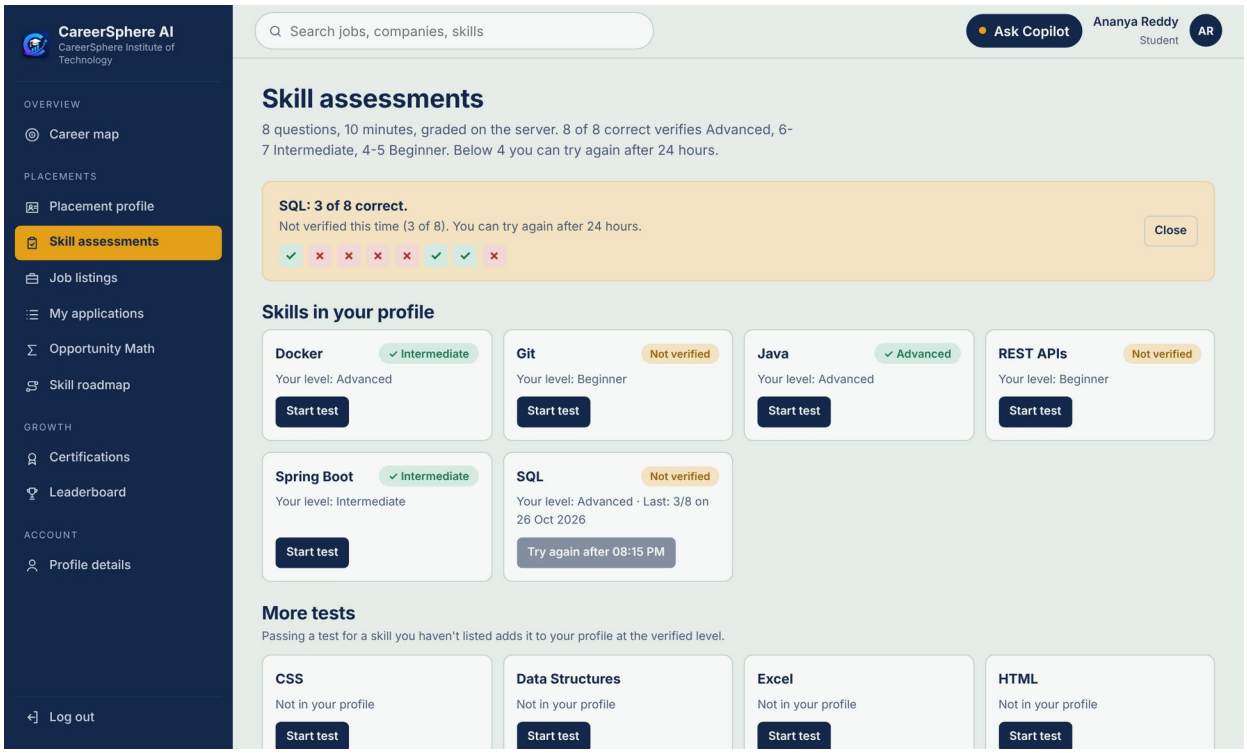



**Fig 6.13 Assessment result: fewer than 4 correct verifies nothing; retake after 24 hours**

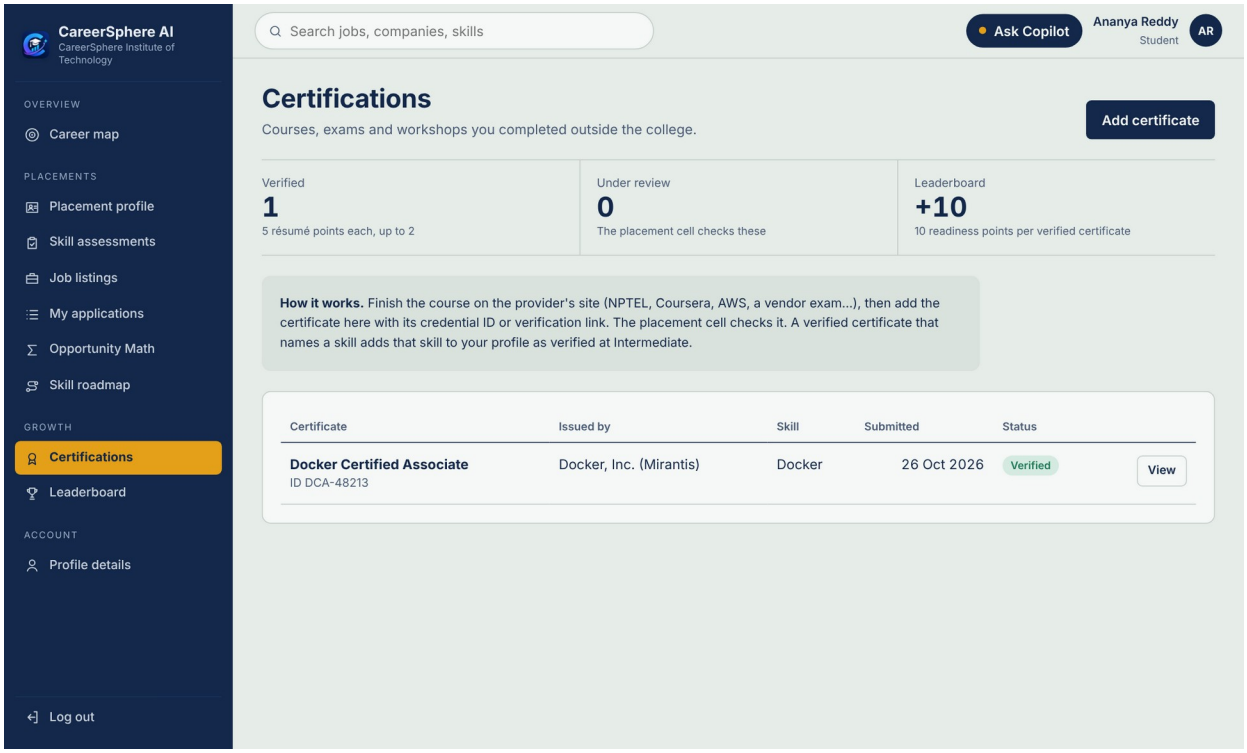



**Fig 6.14 Certifications: a verified certificate and how verification works**

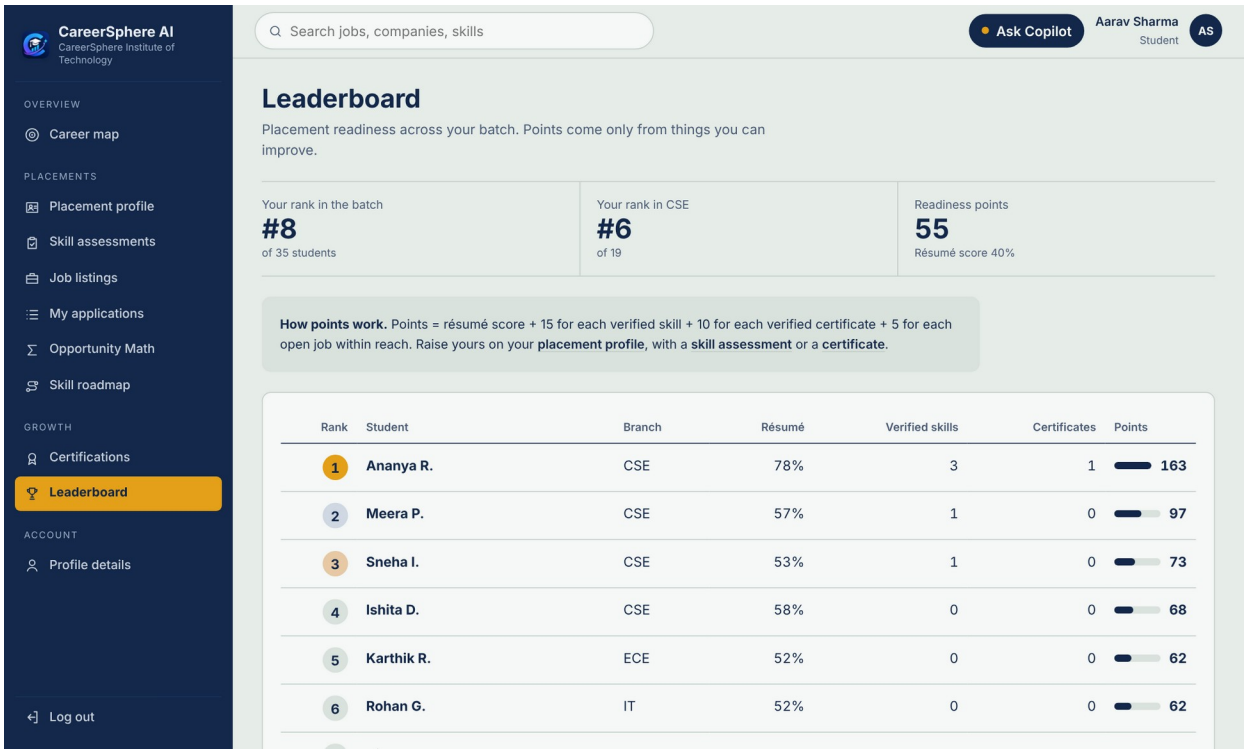



**Fig 6.15 Readiness leaderboard**

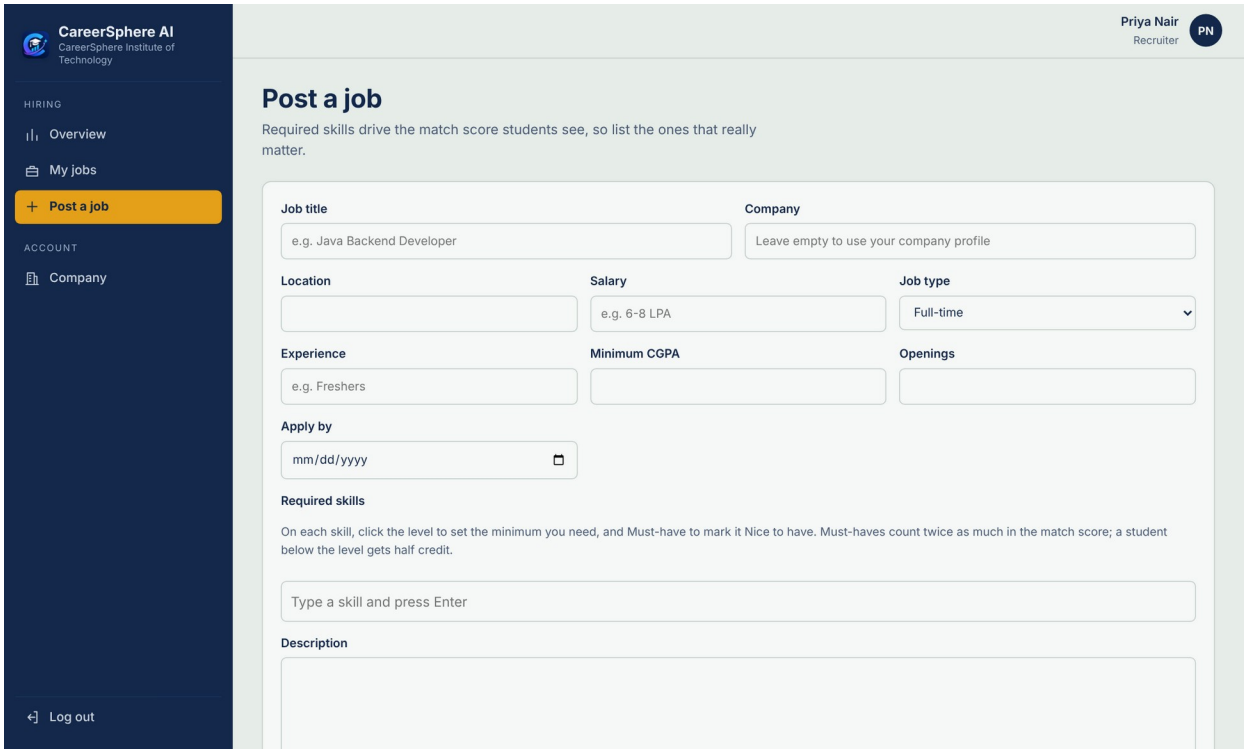



**Fig 6.16 Recruiter: posting a job with skill levels and must-have or nice-to-have flags**

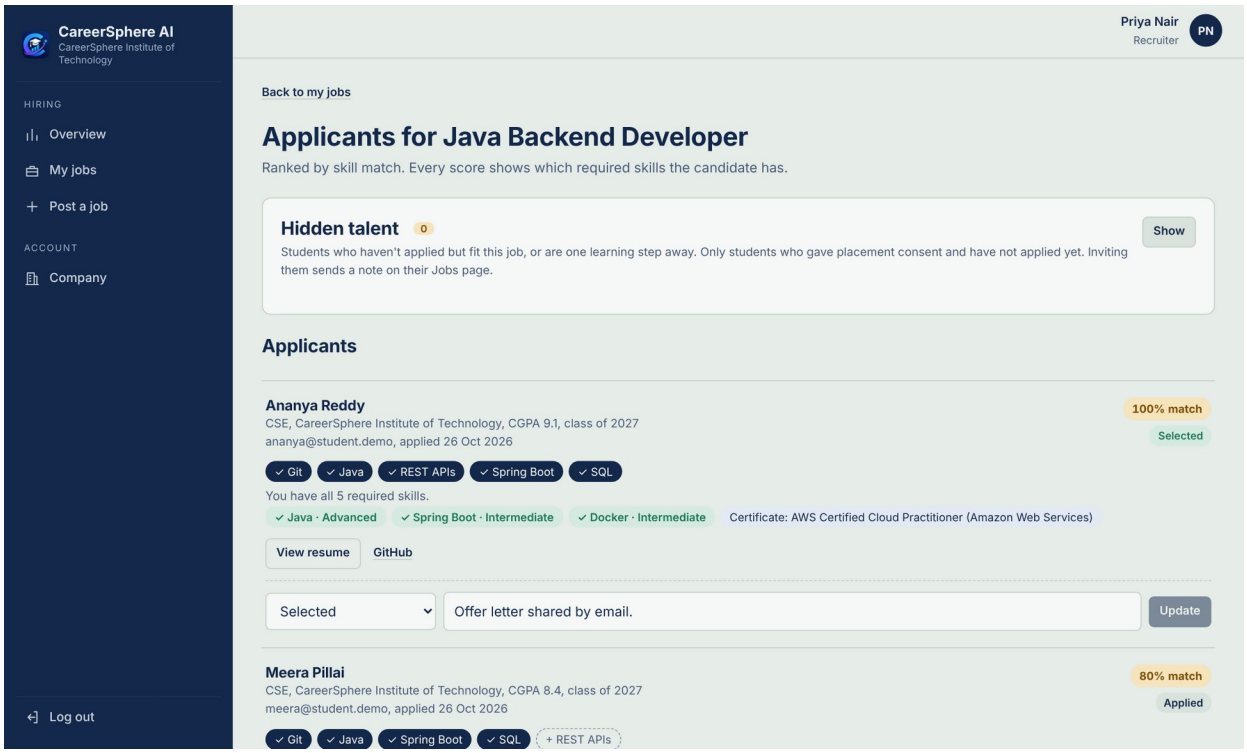



**Fig 6.17 Recruiter: applicants ranked by match, with verified skills**

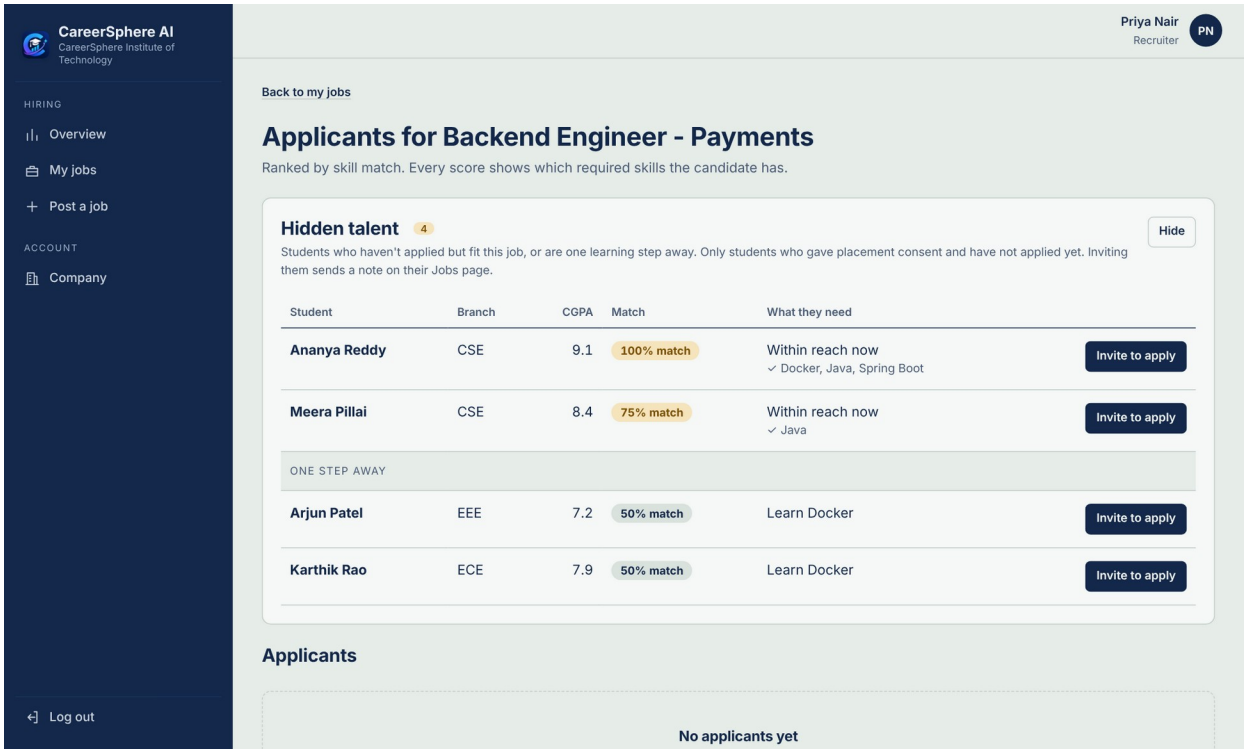



**Fig 6.18 Recruiter: Hidden Talent for a job, with invitations**

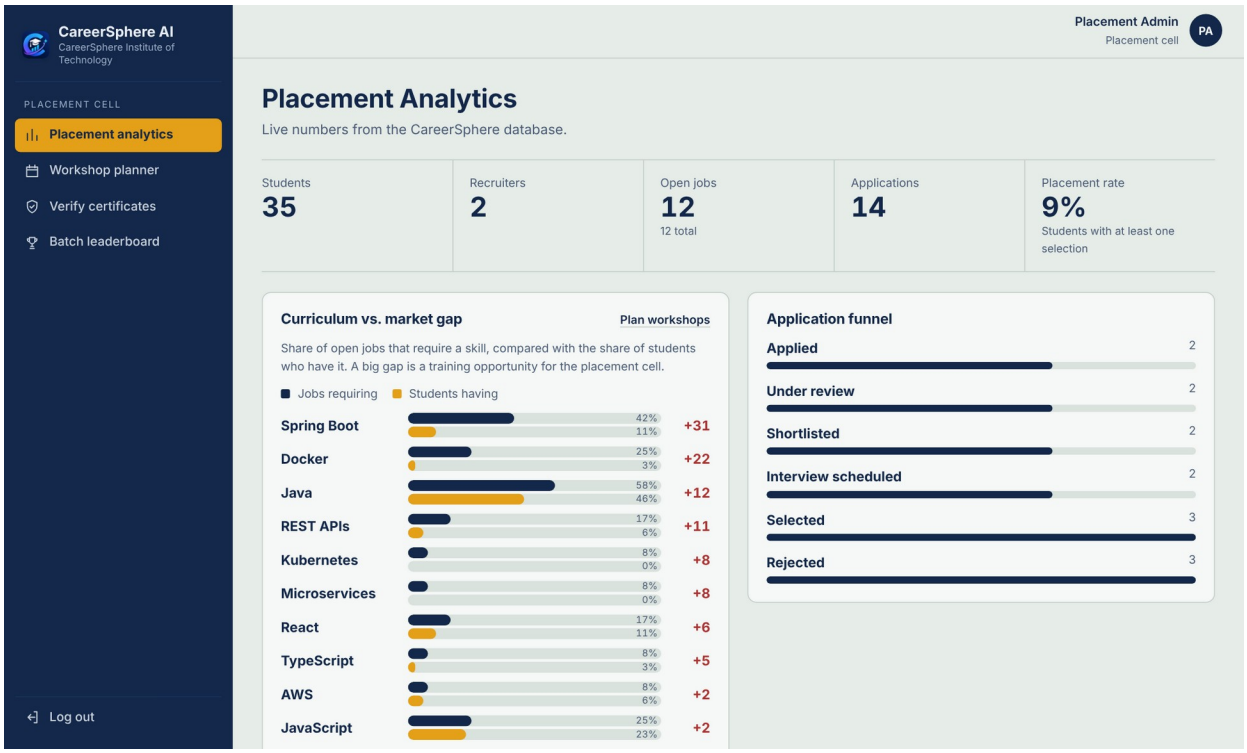



**Fig 6.19 Placement cell: analytics and the curriculum–market skill gap**

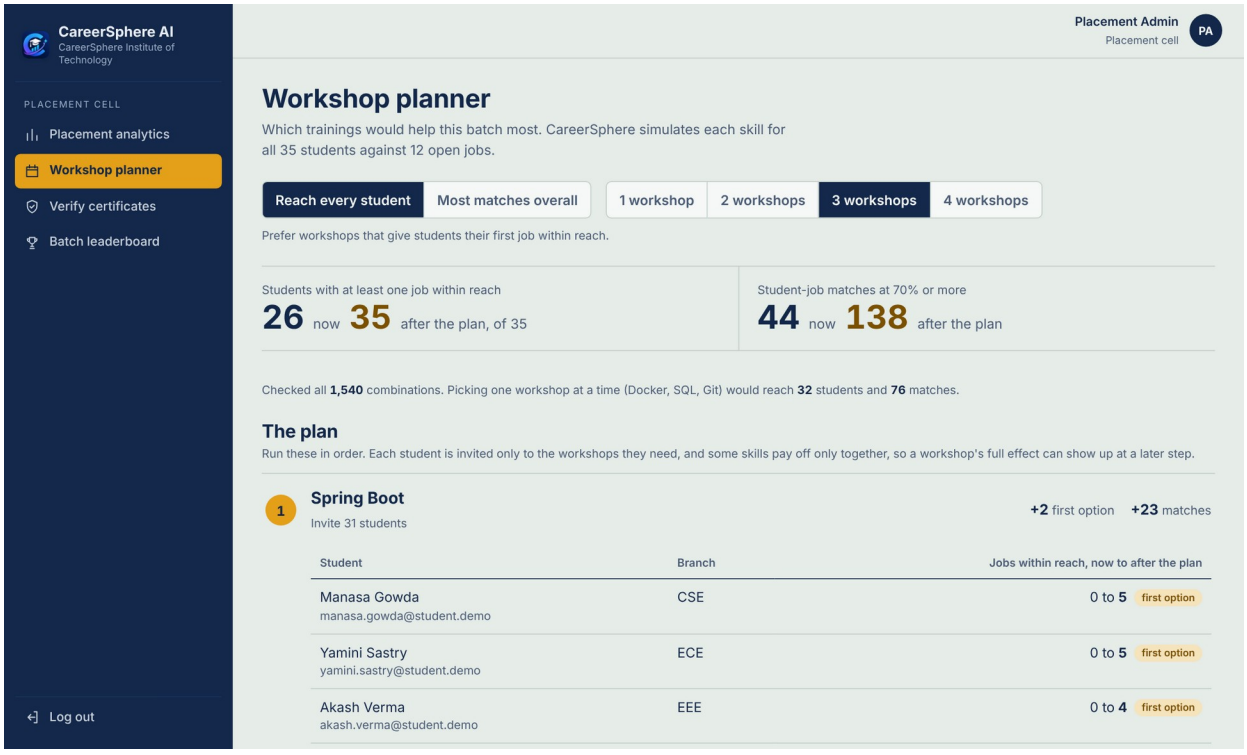



**Fig 6.20 Placement cell: Workshop Planner for three workshops**

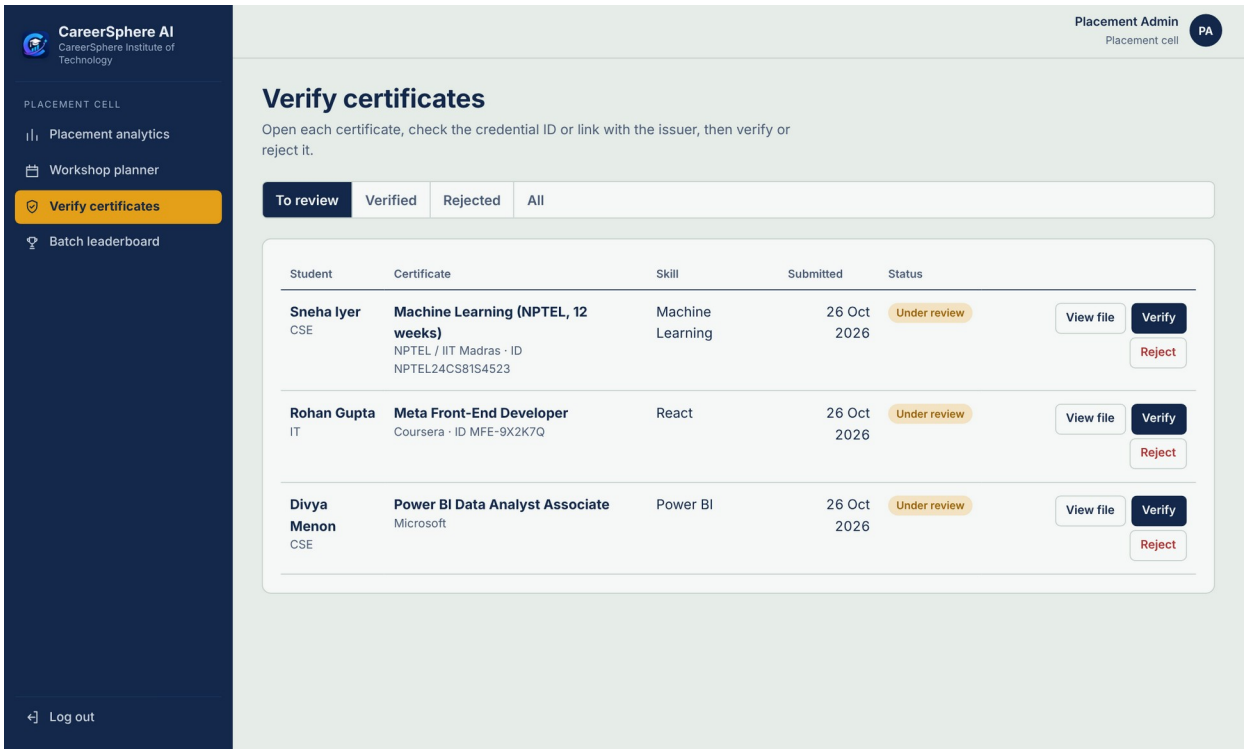



**Fig 6.21 Placement cell: certificates waiting for verification**

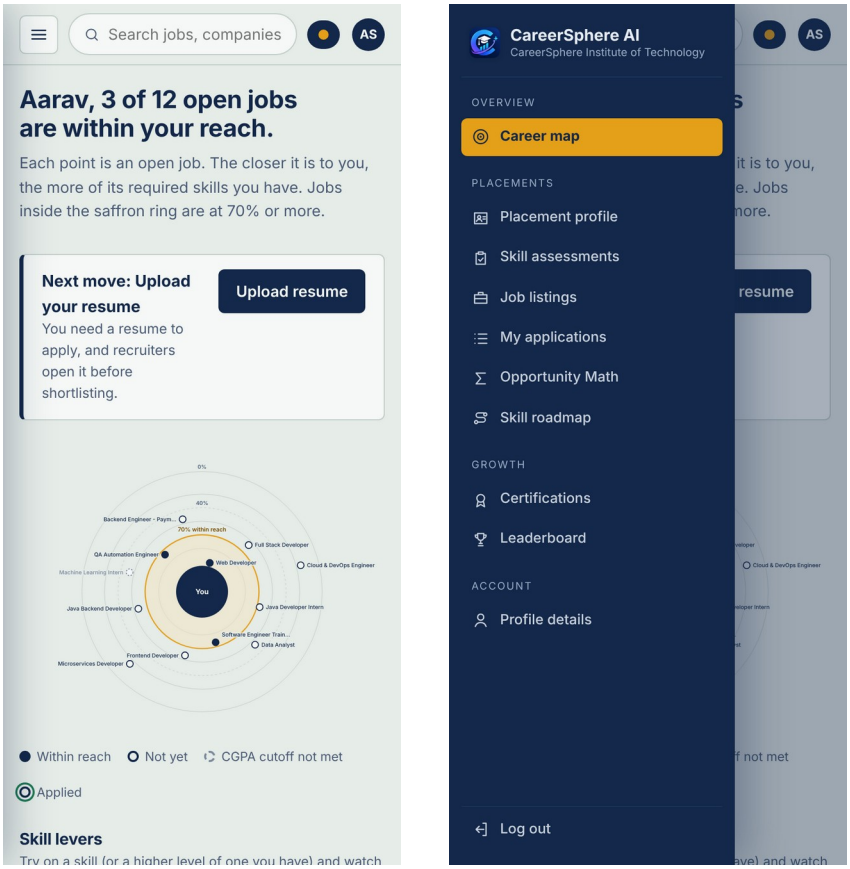



**Fig 6.22 The same app on a phone: Career Map and the slide-in menu**

### 6.5 Running on Different Platforms

- **Terminal (Mac):** `./run.sh` starts MySQL, the backend and the web app, each in its own window, and opens the browser; `./stop.sh` stops them.
- **VS Code:** `code .` in the project folder, then Cmd + Shift + B runs the task “CareerSphere: start everything”; the Run and Debug panel debugs the Java backend and the React app.
- **Xcode:** `CareerSphere.xcodeproj` builds a SwiftUI app for Mac and iPhone that shows the web app in a WKWebView; a build step starts the servers.
- **Android Studio:** the `android` folder is a Kotlin app that shows the web app in a WebView at `10.0.2.2:5173`, with file upload including video capture.
- **Online:** one Docker image builds the web app and puts it inside the backend jar; Docker Compose runs it with MySQL, and Railway hosts it over HTTPS.

# CHAPTER-7

# RESULTS AND DISCUSSION

## 7.1 Experimental Results

All results use the fictional demo batch: 35 students from four branches with 3.49 skills on average, and 12 open jobs from two companies requiring 4.33 skills on average, 4 of them with a CGPA cutoff. There are 22 candidate skills and 420 student–job pairs. Scores are 0% for 144 pairs, 1–39% for 141, 40–69% for 91, 70–99% for 30 and 100% for 14. Today 26 students have at least one job within reach, with 44 matches in total.

### 7.1.1 Next skill and the Skill Roadmap

Table 7.1 compares the top Opportunity Math action with the *demand* heuristic, the missing skill that most open jobs ask for. Table 7.2 and Fig 7.1 add the Skill Roadmap with two and three steps.

**Table 7.1 Recommended next skill, 35 students**

| Measure | Demand heuristic | Opportunity Math |
|---|---|---|
| Mean jobs brought within reach | 1.03 | **1.43** |
| Total jobs brought within reach | 36 | **50** |
| Students whose recommendation unlocks nothing | 20 | **7** |
| Students with a different recommendation | – | 13 |

**Table 7.2 Skill Roadmap, 35 students**

| Steps allowed | Mean new jobs within reach | Students helped |
|---|---|---|
| 1 (Opportunity Math) | 1.43 | 28 |
| up to 2 | 2.89 | **35** |
| up to 3 | **4.71** | **35** |

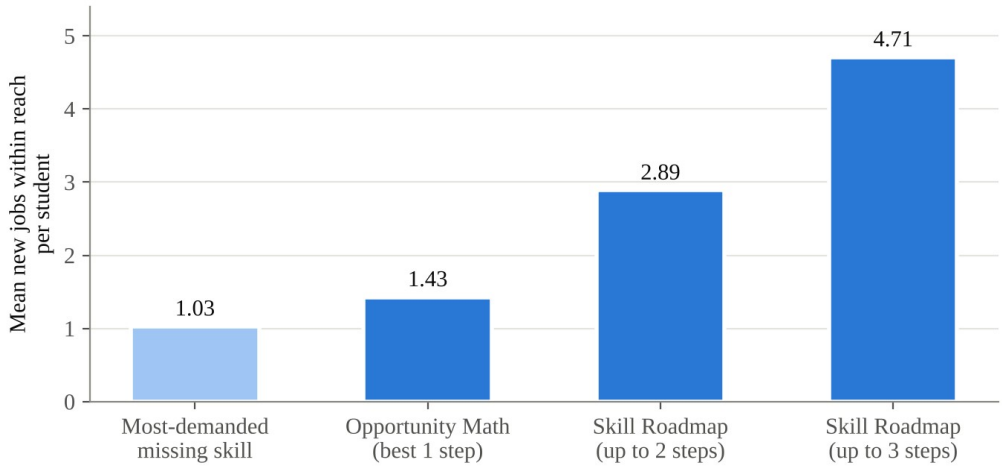



**Fig 7.1 Mean new jobs within reach per student for each way of choosing what to learn**

### 7.1.2 Workshop planning for the batch

Exact search is compared with three baselines for k = 1 to 4 workshops: greedy one-at-a-time selection under the same goal, the top-k skills by job demand, and the top-k skills by demand–supply gap. For the two top-k baselines every student lacking a chosen skill attends (Table 7.3, Fig 7.2). Table 7.4 and Fig 7.3 give the planner runtime, the median of nine calls.

**Table 7.3 Workshop plans: students reached / matches (Reach goal)**

| k | Exact | Greedy | Top-k demand | Top-k gap |
|---|---|---|---|---|
| 1 | **29** / 56 | 29 / 56 | 26 / 44 | 28 / 67 |
| 2 | **33** / 65 | 31 / 64 | 31 / 71 | 31 / 79 |
| 3 | **35** / **138** | 32 / 76 | 35 / 107 | 33 / 99 |
| 4 | **35** / **198** | 33 / 81 | 35 / **198** | 34 / 142 |

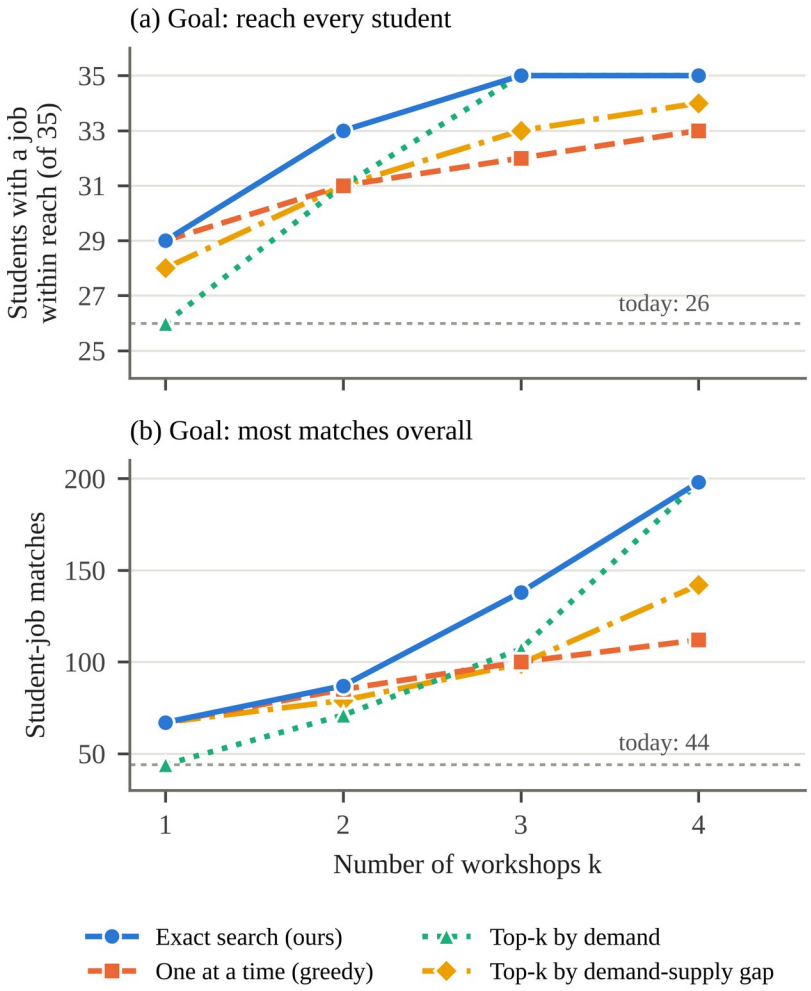



**Fig 7.2 Outcome of each planning method for k = 1 to 4 workshops; dotted line: today. (a) Reach goal; (b) Matches goal**

**Table 7.4 Planner runtime, median of 9 calls**

| k | Combinations | Time (ms) |
|---|---|---|
| 1 | 22 | 0.9 |
| 2 | 231 | 1.9 |
| 3 | 1,540 | 6.5 |
| 4 | 7,315 | 29.3 |
| 5 | 26,334 | 81.7 |
| 6 | 74,613 | 228.9 |

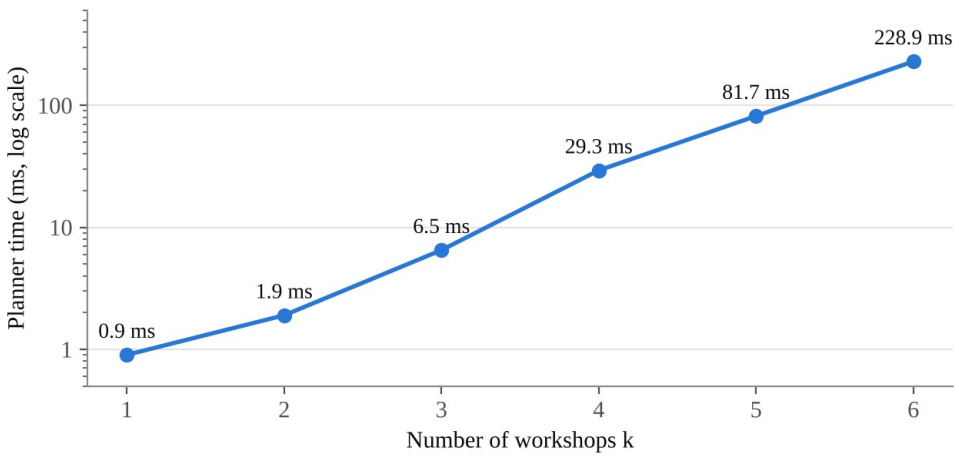



**Fig 7.3 Exact Workshop Planner runtime against the number of workshops (logarithmic scale)**

### 7.1.3 Hidden Talent and the résumé skill finder

Over the 12 jobs, Hidden Talent finds 10 cases of a consented student within reach of a job they have not applied to, and 9 more one step away, against 14 applications actually made by the 10 consented students (Fig 7.4). For “Backend Engineer – Payments” it lists two students within reach and two who need only Docker. On the demo résumé the finder reports six further skills found in project and achievement lines (Spring Boot, REST APIs, MySQL, React, Data Structures and Pandas), each with its quoted line; adding them raises the student’s jobs within reach from 3 to 8.

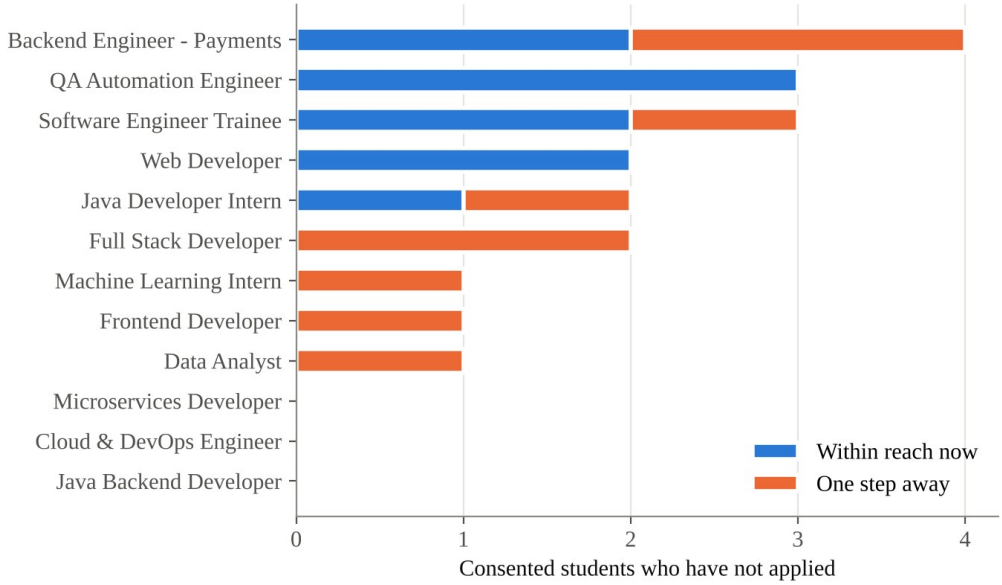



**Fig 7.4 Hidden Talent per job: consented students who have not applied, within reach or one step away**

## 7.2 Discussion and Evaluation of the Research Questions

- **RQ1 – Next skill.** Opportunity Math brings 1.43 jobs per student within reach against 1.03 for the demand heuristic. For 20 of 35 students the most-demanded missing skill would not bring a single job within reach. For the remaining 7 students no single skill helps at all, because every job is two or more skills away.
- **RQ2 – Multi-step plans.** Allowing two steps doubles the gain (2.89) and three steps more than triples it (4.71). All 7 students without a useful single skill gain at least one job with two steps; one of them gains nothing from any one skill but three jobs from Java and SQL together. Of the 65 multi-step bundles shown, 59 are marked *works together*, so the value of the extra steps comes mostly from combining skills. The search checks 1,083 bundles per student on average and runs on every page load.
- **RQ3 – Batch planning.** With three workshops exact search selects Spring Boot, Java and Git and reaches all 35 students with 138 matches, while greedy selects Docker, SQL and Git and reaches 32 with 76. Greedy falls behind even the top-k demand baseline from k = 3, because Java helps few students on its own and pays off only together with Spring Boot and Git. Exact search takes 6.5 ms for k = 3 and stays under a quarter of a second up to k = 6.
- **RQ4 – Recruiter reach.** Hidden Talent surfaces 19 fitting or nearly fitting consented students across the 12 jobs that a recruiter would not otherwise see, without showing any student who did not give consent.

## 7.3 Comparison with Existing Approaches

Table 7.5 places CareerSphere next to the main kinds of approach reviewed in Chapter 2.

**Table 7.5 Comparison of existing approaches with CareerSphere AI**

| Dimension | Learned fit / prediction models [2], [3] | CareerSphere AI |
|---|---|---|
| Data needed | Large labelled histories of past placements | Skills, levels and job requirements only |
| Explanation | Post-hoc or none | Exact: matched, missing, level gaps, CGPA |
| What it answers | Fit or placement probability | Which skill(s) to learn, whom to invite, which workshops |
| Skill combinations | Implicit in the model | Searched exactly; synergy flagged |
| Batch planning | Not addressed | Exact Workshop Planner, greedy shown for comparison |
| AI output | Unchecked text where used | Tools over own data; evidence-checked résumé skills |
| Verification of claims | Self-reported | Server-graded tests and checked certificates |

**Limitations.** All results are on fictional data, so a study over a real placement season is needed. The weights, half credit and threshold are design choices; unverified levels are self-reported; the 120-question bank supports screening rather than certification; and the planner assumes every invitee learns the skill and ignores capacity and cost. A human evaluation kit is ready (60 student–job pairs labelled blind by three people, agreement by Fleiss' κ [15], comparison with keyword matching), but the labelling is not yet complete.

# CHAPTER-8

# CONCLUSION AND FUTURE SCOPE

## 8.1 Conclusions and Research Contributions

This project built CareerSphere AI, a complete campus placement platform for students, recruiters and the placement cell around one transparent, level-aware match rule. The main conclusions are the following.

1. **Actionable guidance beats skill-gap lists.** Opportunity Math brings 1.43 jobs per student within reach against 1.03 for the most-demanded missing skill.
2. **Skills pay off together.** A three-step Skill Roadmap more than triples what the best single skill achieves (4.71 against 1.43 jobs) and helps every student, including the 7 for whom no single skill helps.
3. **Exact search is practical at college scale.** Three exactly chosen workshops reach all 35 students where greedy selection reaches 32, in 6.5 ms.
4. **Reverse matching widens the pool.** Hidden Talent shows recruiters 10 fitting and 9 nearly fitting consented students who did not apply.
5. **Grounded AI is usable and safe.** The Copilot answers only through tools over the student's own data, and the résumé finder keeps only skills whose evidence is in the résumé; both work without an AI key.

The system runs on a laptop with one command, from Xcode, VS Code and Android Studio and online with Docker, and is covered by 39 unit tests and 17 end-to-end API tests.

## 8.2 Future Scope

1. **Human evaluation and a real season.** Complete the labelling study and measure the effect on a real placement season.
2. **Learned parameters.** Learn the weights and the threshold from past outcomes, and calibrate a larger question bank by item difficulty.
3. **Capacity-aware planning.** Plan workshops with capacities and costs as an integer program, which also scales beyond exhaustive search.
4. **Effort in hours.** Measure roadmap effort in hours, with links to specific courses for each step.
5. **Notifications and wider use.** Email and push notifications for status changes and invitations, interview scheduling with calendars, and separate tenants for several colleges.

# REFERENCES


[1] C. de Ruijt and S. Bhulai, “Job recommender systems: A review,” arXiv:2111.13576, 2021.

[2] C. Zhu, H. Zhu, H. Xiong, C. Ma, F. Xie, P. Ding, and P. Li, “Person-job fit: Adapting the right talent for the right job with joint representation learning,” *ACM Trans. Manage. Inf. Syst.*, vol. 9, no. 3, pp. 1–17, 2018.

[3] S. Alfana, S. K. R. Jammalamadaka, V. C. Prakash, and B. T. Reddy, “Cognitive and academic-based probability models for predicting campus placements,” *IAES Int. J. Artif. Intell.*, vol. 11, no. 4, pp. 1239–1251, Dec. 2022, doi: 10.11591/ijai.v11.i4.pp1239-1251.

[4] Y. Zhang and X. Chen, “Explainable recommendation: A survey and new perspectives,” *Found. Trends Inf. Retr.*, vol. 14, no. 1, pp. 1–101, 2020, doi: 10.1561/1500000066.

[5] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, “An analysis of approximations for maximizing submodular set functions—I,” *Math. Program.*, vol. 14, pp. 265–294, 1978.

[6] U. Feige, “A threshold of ln n for approximating set cover,” *J. ACM*, vol. 45, no. 4, pp. 634–652, 1998.

[7] A. Krause and D. Golovin, “Submodular function maximization,” in *Tractability: Practical Approaches to Hard Problems*, L. Bordeaux, Y. Hamadi, and P. Kohli, Eds. Cambridge, U.K.: Cambridge Univ. Press, 2014, pp. 71–104.

[8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. NeurIPS*, 2023.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.

[10] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[11] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling large language models to generate text with citations,” in *Proc. EMNLP*, 2023, pp. 6465–6488, doi: 10.18653/v1/2023.emnlp-main.398.

[12] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in *Proc. Conf. Fairness, Accountability, and Transparency (FAT)**, 2020, pp. 469–481.

[13] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.

[14] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, 1999, pp. 81–91.

[15] J. L. Fleiss, “Measuring nominal scale agreement among many raters,” *Psychol. Bull.*, vol. 76, no. 5, pp. 378–382, 1971.

[16] Google, “Function calling with the Gemini API,” Google AI for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs/function-calling (accessed Oct. 4, 2026).

[17] VMware Tanzu, “Spring Boot Reference Documentation,” version 3.3. [Online]. Available: https://docs.spring.io/spring-boot/

[18] Meta Open Source, “React documentation.” [Online]. Available: https://react.dev/

[19] Oracle, “MySQL 8.0 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[20] OWASP Foundation, “OWASP Top 10: The ten most critical web application security risks,” 2021. [Online]. Available: https://owasp.org/Top10/

# APPENDIX
# SETUP COMMANDS, DEMO SCRIPT AND API LIST

The commands below set up the project on a Mac, run it, reset the demo data and run the tests. The data lives in the MySQL database `careersphere` on `localhost:3306` (user `root`, no password by default); uploaded files are saved in `backend/uploads/`.

**Listing A.1 One-time setup on a Mac**

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install openjdk@17 maven node mysql
sudo ln -sfn $(brew --prefix)/opt/openjdk@17/libexec/openjdk.jdk \
     /Library/Java/JavaVirtualMachines/openjdk-17.jdk
brew services start mysql
```

**Listing A.2 Running the project**

```
cd careersphere
code .                 # then press Cmd + Shift + B in VS Code ("CareerSphere: start everything")
# or, by hand in two terminals:
cd backend  && mvn spring-boot:run           # API on http://localhost:8080
cd frontend && npm install && npm run dev    # web app on http://localhost:5173
./run.sh               # everything with one command;  ./stop.sh  stops it
```

**Listing A.3 Reset, tests, Docker and the optional AI key**

```
mysql -u root -e "DROP DATABASE careersphere"   # reset: the demo data is rebuilt on next start
cd backend && mvn test                           # 39 unit tests and 17 API tests
docker compose up --build                        # app and MySQL on http://localhost:8080
export GEMINI_API_KEY=your-key                   # optional; only in ~/.zshrc, never in the project
```

**Table A.1 Demo accounts**

| Role | Email | Password |
|---|---|---|
| Student | student@careersphere.ai | student123 |
| Recruiter | recruiter@careersphere.ai | recruiter123 |
| Placement cell | admin@careersphere.ai | admin123 |
| Other students | ananya@student.demo, rohan@student.demo, ... | student123 |

**Demo script (about 10 minutes).** (1) Log in as the student: the Career Map shows 3 of 12 jobs within reach; switch on the Spring Boot lever and four jobs move inside the ring. (2) Opportunity Math puts Spring Boot first with +4 jobs; the Skill Roadmap gives +6 with three steps. (3) On the placement profile, upload `demo/Aarav_Sharma_Resume.pdf`, accept the six suggested skills and give consent: jobs within reach rise from 3 to 8. (4) Take an assessment and show that it is graded on the server. (5) Ask the Copilot “Which skill should I learn next?”. (6) Apply to a job; as the

recruiter, open its applicants and change a status; open Hidden Talent on “Backend Engineer – Payments” and invite Ananya. (7) As Ananya, show the invitation on the Jobs page. (8) As the placement cell, verify a certificate and plan three workshops: all 35 students reached, against 32 one at a time.

**Table A.2 All REST endpoints**

| Prefix | Endpoints |
|---|---|
| /api (public) | GET /health; POST /auth/register; POST /auth/login; GET /auth/me; GET /skills |
| /api/student | GET /dashboard, /opportunities, /what-if; GET, PUT /profile; POST /profile/skills; POST, GET, DELETE /resume; GET /resume/file; POST /resume/skills; GET /jobs, /jobs/{id}; POST, GET /applications; GET /roadmap, /invitations, /readiness; PUT /consent; GET /leaderboard; POST, GET, DELETE /video; GET /video/file; GET /assessments; POST /assessments/start, /assessments/{id}/submit; POST, GET /certificates; DELETE /certificates/{id}; GET /certificates/{id}/file |
| /api/student/copilot | GET /status; POST (ask) |
| /api/recruiter | GET /dashboard; GET, PUT /profile; GET, POST /jobs; PUT, DELETE /jobs/{id}; PATCH /jobs/{id}/status; GET /jobs/{id}/applicants; PATCH /applications/{id}/status; GET /applications/{id}/resume, /applications/{id}/video; GET /jobs/{id}/talent; POST /jobs/{id}/invite/{studentId} |
| /api/admin | GET /stats, /users, /workshops, /certificates; PATCH /certificates/{id}; GET /certificates/{id}/file; GET /leaderboard |